Carpeta del Proyecto — ProgMesas

Desarrollo de un Sistema Web para Gestión de Programas de Examen Integrado con Google Drive (ProgMesas)

Alan Buzzatto y Valentino Rodriguez

Escuela de Educación Técnica N.° 2 "Independencia"

Proyecto de Práctica Profesional (PPP) — 7.° año, 1.ª división, Técnico en Computación

Profesor: Juan Roude

27 de octubre de 2026

Índice

1. Presentación del proyecto..... 3
2. Objetivos..... 4
Objetivo general..... 4
Objetivos específicos..... 4
3. Descripción del dominio del sistema..... 5
Conceptos del dominio..... 5
Cursos de la escuela..... 5
Actores..... 6
Reglas del dominio..... 6
4. Límites del sistema..... 7
Marco legal..... 7
5. Requerimientos funcionales y no funcionales..... 9
Requerimientos funcionales..... 9
Requerimientos no funcionales..... 10
6. Diagrama de Gantt..... 12
7. Casos de uso..... 14
Descripción de CU07: Subir o reemplazar programa..... 15
Descripción de CU06: Vincular cuenta con la lista..... 16
8. Diagrama Entidad-Relación y diccionario de datos..... 17
Diccionario de datos..... 17
9. Anexos..... 21
Anexo A: Relevamiento y estudio de factibilidad..... 21
Anexo B: Código..... 21
Anexo C: Documentación externa (guía de uso)..... 51
Anexo D: Documentación interna..... 54
10. Referencias..... 72

1. Presentación del proyecto

ProgMesas es un sistema web que centraliza los programas de examen de la Escuela Técnica N.° 2 "Independencia": los docentes los cargan en PDF, la asesoría pedagógica controla qué falta y cualquier alumno o familia los consulta sin iniciar sesión.

Antes de cada mesa de examen, la escuela necesita reunir un programa por cada materia y curso. Hasta ahora los archivos estaban dispersos, era difícil encontrar la versión actualizada, no había una organización por año, curso y materia, y no se podía seguir qué programas faltaban.

ProgMesas resuelve ese problema con tres piezas:

- **Carga ordenada**: cada docente puede subir el PDF del programa de cualquier curso y materia de la lista oficial de horarios (473 asignaciones); los suyos aparecen primero.
- **Control de entregas**: la asesoría tiene un tablero con el porcentaje de cobertura por curso, la fecha límite, el listado de docentes con programas pendientes y un registro de actividad.
- **Consulta pública**: alumnos, familias y preceptores buscan por año, curso o materia, ven el PDF en el navegador y descargan todos los programas de un curso en un ZIP.

El sistema se integra con el campus virtual de la escuela: cada persona inicia sesión en el campus y entra a ProgMesas desde el módulo de programas de examen, sin otra contraseña. Además, ProgMesas copia el diseño del sistema de asistencia de la escuela (PROJECT_ASIST) para que la comunidad lo reconozca como parte de la misma plataforma. Los archivos se guardan en el Google Drive institucional.

El proyecto fue desarrollado por Alan Buzzatto y Valentino Rodriguez en el marco del Proyecto de Práctica Profesional de 7.° año. El código es público en GitHub (alan4352/progMesas) y cuenta con una demostración navegable en GitHub Pages.

2. Objetivos

Objetivo general

Desarrollar un sistema web que permita a la Escuela Técnica N.° 2 "Independencia" reunir, controlar y publicar los programas de examen de todas las materias y cursos en un solo lugar, integrado al campus virtual de la escuela.

Objetivos específicos

1. Permitir que cada docente cargue, reemplace o reutilice en PDF el programa de cualquier curso y materia, sin depender de la aprobación manual de otra persona.
2. Dar a la asesoría pedagógica un tablero que muestre la cobertura por curso y por docente, los programas pendientes y atrasados respecto de una fecha límite.
3. Publicar los programas para que alumnos y familias los consulten sin cuenta, incluidos los de años anteriores, y descarguen los de un curso en un solo archivo.
4. Conservar el historial de versiones de cada programa y un registro de quién hizo cada cambio.
5. Guardar los archivos en el Google Drive de la escuela, con respaldo en el servidor y copias de seguridad diarias.
6. Usar el inicio de sesión del campus virtual de la escuela y el diseño de su sistema de asistencia (PROJECT_ASIST), para que los usuarios no tengan que aprender una herramienta distinta.
7. Asegurar la calidad con pruebas automáticas de la API y de la interfaz, incluida la accesibilidad (WCAG 2.1 AA).

3. Descripción del dominio del sistema

El dominio de ProgMesas es la gestión de los programas de examen de una escuela secundaria técnica: el documento que cada docente entrega por materia y curso, y que los alumnos usan para preparar las mesas de examen (regulares, previas y libres).

Conceptos del dominio

Concepto	Qué es en la escuela	Cómo lo representa ProgMesas
Programa de examen	Documento con los contenidos y la bibliografía que se evalúan en una materia	Un PDF asociado a una asignación y a un año lectivo; además del programa común, cada asignación puede tener un programa libre, para los alumnos que rinden en condición de libres
Asignación	Una materia dictada en un curso por uno o más docentes	Fila de la lista oficial (473 asignaciones), con id curso + materia, p. ej. `2-1_matematica`
Curso	Año y división (por ejemplo, 2.° 1.ª), con su turno	Campo de la asignación y filtro de búsqueda
Año lectivo	Ciclo escolar al que pertenece el programa	Campo del programa; permite consultar años anteriores
Fecha límite	Plazo que fija la asesoría para entregar los programas	Ajuste por año; marca atrasados y entregas fuera de término
Versión	Programa reemplazado por uno corregido	Historial con el PDF anterior, quién lo reemplazó y cuándo

Cursos de la escuela

La escuela tiene 29 cursos. Desde 4.° año, cada división corresponde a una especialidad (TC, TG o TAE).

Año	Cursos
1.°	1 1, 1 2, 1 3, 1 4, 1 5
2.°	2 1, 2 2, 2 3, 2 4
3.°	3 1, 3 2, 3 3
4.°	4 1 TC, 4 1 TG, 4 1 TAE, 4 2 TAE
5.°	5 1 TC, 5 1 TG, 5 1 TAE, 5 2 TAE
6.°	6 1 TC, 6 1 TG, 6 1 TAE
7.°	7 1 TC, 7 1 TG, 7 1 TAE

Actores

Actor	Rol en la escuela	Qué hace en el sistema
Alumno / familia / visitante	Consulta los programas para rendir	Busca, abre y descarga programas sin iniciar sesión
Docente	Dicta una o más materias	Carga, reemplaza y reutiliza los programas de sus asignaciones
Asesora pedagógica (en el campus, directivo)	Controla la entrega de programas	Usa el dashboard, fija la fecha límite, corrige vinculaciones, carga o elimina cualquier programa
Consulta (preceptor)	Supervisa	Ve el dashboard y el registro sin modificar nada
Campus virtual de la escuela	Plataforma donde inician sesión alumnos, profesores y directivos	Arma un pase firmado con el usuario y su rol y abre ProgMesas
Google Drive	Almacenamiento institucional	Guarda los PDF en carpetas Año / Curso / Materia

Reglas del dominio

- Matemática de 2.º 1.ª y Matemática de 3.º 2.ª son programas distintos, aunque la materia se llame igual: el control se hace por asignación, no por nombre de materia.
- Cada asignación tiene un solo programa vigente por año, más un programa libre opcional; al reemplazar cualquiera de los dos, el anterior pasa al historial. Para la cobertura, la asignación cuenta como entregada con el programa común o con el libre.
- Un docente puede cargar el programa de cualquier asignación de la lista oficial; cada carga queda registrada con su cuenta y, si reemplaza un programa, la versión anterior se conserva.
- Las cuentas llegan verificadas desde el campus virtual: la cuenta se vincula sola con el nombre de la lista si coincide, o el docente lo elige (es opcional: sin vincular, figura con el nombre de su cuenta del campus). La asesora puede corregir o anular esa vinculación.
- Los programas de años anteriores siguen publicados, porque los alumnos que adeudan materias los necesitan.

4. Límites del sistema

ProgMesas cubre el ciclo de vida del programa de examen, desde la carga hasta la consulta; no gestiona usuarios, notas ni la organización de las mesas.

Dentro del alcance

- Carga, reemplazo, reutilización y eliminación de programas en PDF, comunes y libres.
- Control de cobertura por año, curso y docente, con fecha límite y recordatorios por correo (se abre el cliente de correo del usuario).
- Consulta pública, visor de PDF, enlaces por curso y descarga en ZIP.
- Historial de versiones, registro de actividad y copias de seguridad.
- Almacenamiento en Google Drive con respaldo en el servidor.

Fuera del alcance

- Alta, baja y verificación de usuarios: las hace el campus virtual de la escuela. ProgMesas solo recibe el pase firmado que arma el campus.
- Armado de las mesas de examen: fechas, tribunales, inscripción de alumnos y calificaciones.
- Redacción o corrección del contenido de los programas: el sistema guarda el PDF que entrega el docente.
- Edición de la lista de horarios dentro de la página: la lista oficial se mantiene en una planilla Excel que la asesoría edita.
- Envío automático de correos o notificaciones desde el servidor.
- Formatos distintos de PDF.

Restricciones

- La instalación definitiva es en el servidor de la escuela; Render se usa solo de forma temporal y GitHub Pages solo como demostración.
- Google Drive necesita la autorización de una cuenta de la escuela. Sin ella, los PDF quedan en el servidor.
- Cada PDF puede pesar hasta 10 MB (valor configurable).

Marco legal

Norma	Qué regula	Cómo la respeta ProgMesas
Ley 25.326 de Protección de los Datos Personales	Tratamiento de nombres, correos y otros datos personales	Guarda solo los datos necesarios (nombre, correo, rol), publica una política de privacidad y no permite cargar datos de alumnos en los programas.
Ley 11.723 de Propiedad Intelectual	Derechos de autor sobre obras, incluido el software y el material educativo	La autoría de cada programa sigue siendo de su docente; las condiciones del servicio lo establecen.
Ley 26.206 de Educación Nacional	Marco del sistema educativo	El sistema apoya un proceso institucional de la escuela sin reemplazar sus canales oficiales.

Norma	Qué regula	Cómo la respeta ProgMesas
Ley 26.653 de Accesibilidad de la Información en las Páginas Web	Accesibilidad de los sitios web	La interfaz cumple WCAG 2.1 AA, verificado con pruebas automáticas.

5. Requerimientos funcionales y no funcionales

El sistema cumple 18 requerimientos funcionales y 12 no funcionales; todos están implementados y la mayoría tiene pruebas automáticas (53 de la API y 11 de la interfaz).

Requerimientos funcionales

Código	Requerimiento	Actor
RF01	Iniciar sesión con el campus virtual: el campus arma un pase firmado y abre ProgMesas sin pedir otra contraseña (profesor → docente, directivo → asesora, alumno → alumno). ProgMesas no tiene login propio.	Todos los usuarios con cuenta
RF02	Vincular la cuenta del docente con su nombre en la lista oficial: automáticamente si coincide, o eligiéndolo (opcional: sirve para mostrar sus materias primero).	Docente
RF03	Mostrar al docente todas las asignaciones del año, con las suyas primero, el estado de cada programa y la fecha límite.	Docente
RF04	Subir el programa en PDF de cualquier asignación (curso y materia), para el año lectivo actual, indicando si es el programa libre; el título se genera solo y sólo aclara «libre» en ese caso.	Docente, Asesora
RF05	Reemplazar un programa conservando la versión anterior en el historial.	Docente, Asesora
RF06	Reutilizar el programa del año anterior para la misma asignación (común o libre) sin volver a subirlo.	Docente, Asesora
RF07	Eliminar un programa (el archivo se borra solo si ningún otro programa lo usa).	Asesora
RF08	Buscar programas por año, curso, materia o docente, incluidos los de años anteriores.	Alumno
RF09	Ver el PDF en un visor dentro de la página y descargarlo.	Alumno
RF10	Compartir el enlace de una búsqueda (filtros en la dirección de la página).	Alumno
RF11	Descargar en un ZIP todos los programas de un curso, ordenados por año.	Alumno

Código	Requerimiento	Actor
RF12	Mostrar el dashboard con cobertura del año, avance por curso y por docente, y pendientes.	Asesora, Consulta
RF13	Fijar la fecha límite de entrega de cada año y marcar atrasados y entregas fuera de término.	Asesora
RF14	Exportar pendientes a CSV, copiar los correos de docentes con pendientes y abrir un recordatorio por correo.	Asesora
RF15	Corregir o anular la vinculación de un docente y vincular programas viejos a una asignación.	Asesora
RF16	Registrar cada acción que modifica datos y mostrar el registro de actividad.	Sistema; Asesora y Consulta lo ven
RF17	Guardar los PDF en Google Drive y, si no está disponible, en el servidor, pasándolos a Drive al arrancar, cada 6 horas o con un botón.	Sistema, Asesora
RF18	Hacer una copia de seguridad diaria de la base de datos, los PDF del servidor y la planilla.	Sistema

Requerimientos no funcionales

Código	Tipo	Requerimiento
RNF01	Seguridad	Verificar la firma (HS256 con una clave compartida), el vencimiento (8 horas) y el rol del pase del campus virtual.
RNF02	Seguridad	No guardar usuarios ni contraseñas: el inicio de sesión lo hace el campus y ProgMesas sólo verifica el pase.
RNF03	Seguridad	Aceptar solo PDF válidos (se revisa el contenido) de hasta 10 MB.
RNF04	Seguridad	Escapar todo el contenido cargado por usuarios y bloquear scripts externos con una política CSP.
RNF05	Seguridad	Mantener las credenciales fuera del repositorio y pasarlas por variables de entorno.
RNF06	Usabilidad	Usar el mismo diseño que el sistema de asistencia de la escuela (PROJECT_ASIST), con modo claro y oscuro.
RNF07	Accesibilidad	Cumplir WCAG 2.1 AA en los dos modos, verificado con axe-core.
RNF08	Portabilidad	Funcionar en celulares, tabletas y computadoras (diseño adaptable).

Código	Tipo	Requerimiento
RNF09	Disponibilidad	Seguir funcionando si Google Drive no responde, guardando los PDF en el servidor.
RNF10	Mantenibilidad	Configurar el sistema con variables de entorno, sin tocar el código.
RNF11	Calidad	Ejecutar las pruebas automáticas en cada cambio con GitHub Actions (Node 20 y 22).
RNF12	Privacidad	Publicar la política de privacidad y las condiciones del servicio, y usar Drive solo para los fines declarados.

6. Diagrama de Gantt

El proyecto se realizó en 12 actividades entre el 23 de abril y el 27 de octubre de 2026, día de entrega de la documentación. Cada actividad depende de la anterior: empieza cuando la otra termina (las flechas). La 11 y la 12 se superponen del 23 al 26 de octubre, los únicos días con dos actividades a la vez; la franja de carga de trabajo muestra cuántas actividades hay en curso cada día. Octubre se dibuja ampliado por semana para que se lean sus seis actividades cortas.

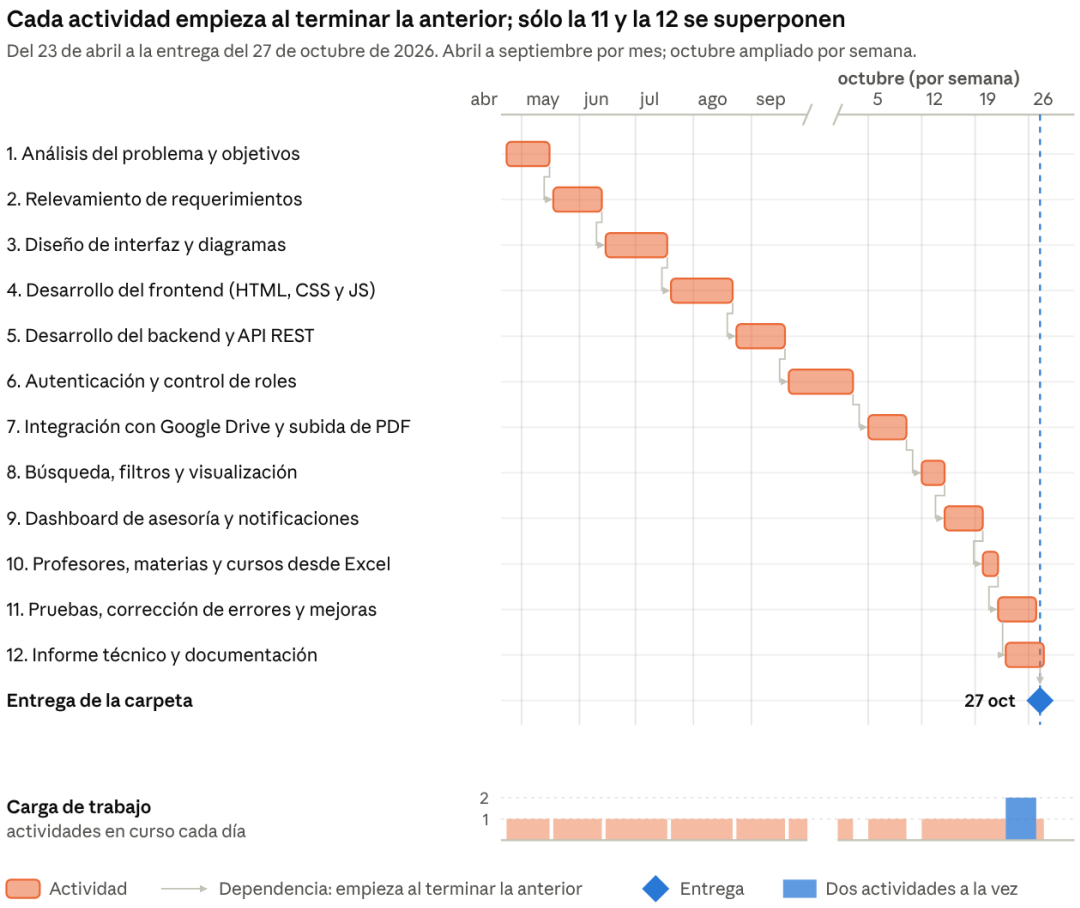


Diagrama de Gantt · 12 actividades, abril a octubre de 2026

N.°	Actividad	Inicio	Fin
1	Análisis del problema y objetivos	23/04/2026	15/05/2026
2	Relevamiento de requerimientos	18/05/2026	12/06/2026
3	Diseño de interfaz y diagramas	15/06/2026	17/07/2026
4	Desarrollo del frontend (HTML, CSS y JS)	20/07/2026	21/08/2026
5	Desarrollo del backend y API REST	24/08/2026	18/09/2026
6	Autenticación y control de roles	21/09/2026	02/10/2026
7	Integración con Google Drive y subida de PDF	05/10/2026	09/10/2026

N.°	Actividad	Inicio	Fin
8	Búsqueda, filtros y visualización de programas	12/10/2026	14/10/2026
9	Dashboard de asesoría y notificaciones	15/10/2026	19/10/2026
10	Organización de profesores, materias y cursos desde Excel	20/10/2026	21/10/2026
11	Pruebas, corrección de errores y mejoras	22/10/2026	26/10/2026
12	Elaboración del informe técnico y documentación	23/10/2026	27/10/2026
—	Entrega de la documentación	27/10/2026	27/10/2026

7. Casos de uso

El sistema tiene 16 casos de uso y 3 actores: alumno, docente y asesora. El docente puede hacer todo lo del alumno, y la asesora todo lo del docente para cualquier materia.



Diagrama de casos de uso · 16 casos, 3 actores

El diagrama sigue la notación UML. Las líneas llenas unen a cada actor con los casos que inicia; el triángulo hueco es la generalización (el docente hereda los casos del alumno y la asesora los del docente). Las flechas punteadas son dependencias: «include», un caso que el caso base siempre usa, y «extend», una opción que se agrega al caso base. El caso central es CU07 (resaltado).

Caso de uso	Actor que lo inicia	Relación
CU01 Buscar programas	Alumno (y por herencia, docente y asesora)	Caso base

Caso de uso	Actor que lo inicia	Relación
CU02 Ver programa en PDF	Alumno	«extend» CU01
CU03 Compartir enlace de búsqueda	Alumno	«extend» CU01
CU04 Descargar ZIP de un curso	Alumno	«extend» CU01
CU05 Iniciar sesión	Docente y asesora	Caso base
CU06 Vincular cuenta con la lista	Docente	«extend» CU05
CU07 Subir o reemplazar programa	Docente y asesora	«include» CU05
CU08 Reutilizar programa anterior	Docente y asesora	«extend» CU07
CU09 Ver versiones anteriores	Docente y asesora	«extend» CU01
CU10 Fijar fecha límite	Asesora	«extend» CU15
CU11 Corregir vinculaciones	Asesora	«extend» CU15
CU12 Eliminar programa	Asesora	«extend» CU01
CU13 Enviar recordatorios	Asesora	«extend» CU15
CU14 Pasar programas a Drive	Asesora	«extend» CU15
CU15 Ver dashboard	Asesora	Caso base
CU16 Ver registro de actividad	Asesora	«extend» CU15

Descripción de CU07: Subir o reemplazar programa

Campo	Detalle
Actores	Docente (principal), Asesora
Precondición	El docente inició sesión desde el campus (no hace falta que esté vinculado con la lista oficial).
Flujo principal	1. El docente elige curso y materia entre todas las asignaciones (las suyas aparecen primero). 2. Si es el programa para alumnos libres, marca «Es el programa libre»; el año lectivo (el actual) y el título se completan solos. 3. Elige el archivo PDF y toca **Guardar programa**. 4. El sistema verifica que sea un PDF válido de hasta 10 MB. 5. Guarda el archivo en Google Drive (Año / Curso / Materia). 6. Registra el programa, anota la acción en el registro y lo publica.
Flujo alternativo A	Ya hay un programa para esa asignación y año: el sistema avisa, el docente confirma y el anterior pasa a Versiones anteriores.
Flujo alternativo B	Google Drive no está disponible: el PDF se guarda en el servidor y pasa a Drive automáticamente cuando vuelve.

Campo	Detalle
Excepción	El archivo no es un PDF válido, supera el tamaño o el año no es el actual: se muestra el error y no se guarda nada.
Postcondición	El programa queda publicado y la cobertura del dashboard se actualiza.

Descripción de CU06: Vincular cuenta con la lista

Campo	Detalle
Actores	Docente
Precondición	El docente inició sesión en el campus virtual y abrió el módulo de programas de examen.
Flujo principal	1. Al entrar, el sistema compara el nombre de la cuenta con la lista oficial (mismas palabras en cualquier orden, sin tildes). 2. Si hay una sola coincidencia libre, vincula la cuenta automáticamente.
Flujo alternativo	Si no hay coincidencia, el docente escribe su apellido, elige su nombre y toca **Confirmar**; queda vinculado en el momento.
Excepción	El nombre ya está tomado por otra cuenta: no se permite elegirlo.
Postcondición	Las asignaciones del docente aparecen primero al cargar y figura con su nombre de la lista; la asesora puede corregir o anular la vinculación.

8. Diagrama Entidad-Relación y diccionario de datos

La base de datos SQLite tiene cinco tablas (ProgMesas no guarda usuarios ni contraseñas: vienen del campus); la lista oficial de asignaciones vive en una planilla Excel y se relaciona con los programas por el campo `asignacion_id`.

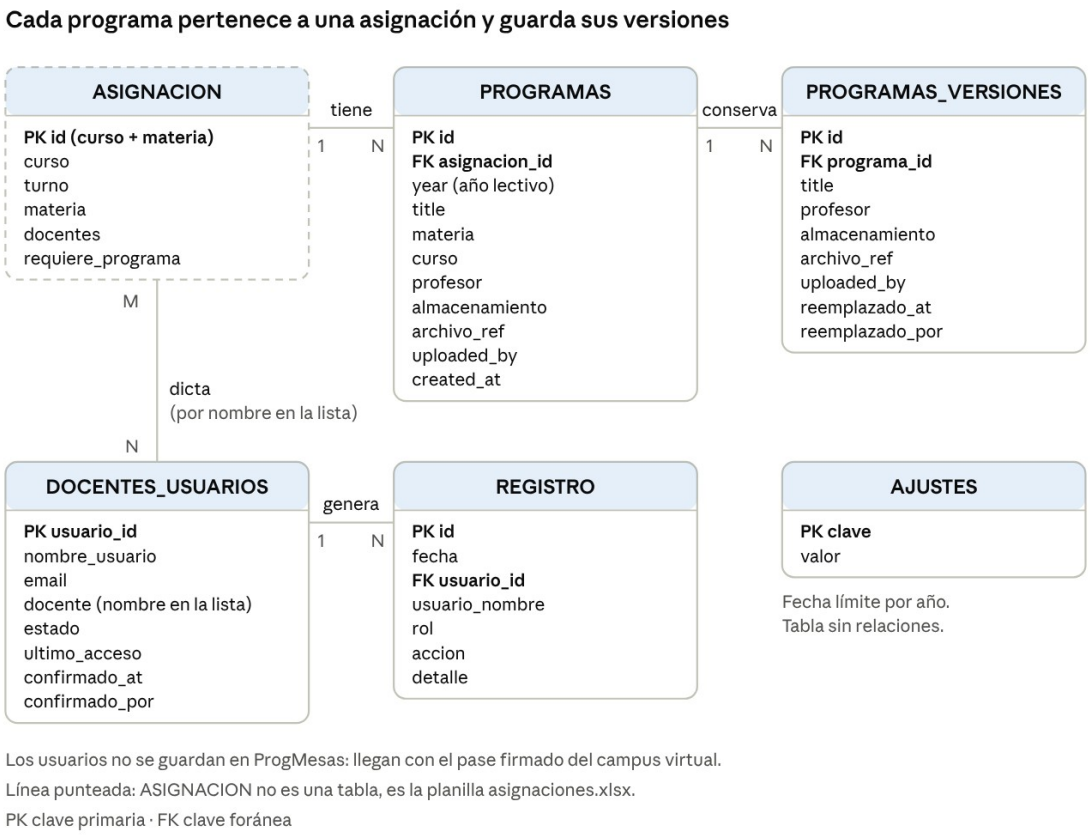


Diagrama entidad-relación · 5 tablas y la planilla de asignaciones

Cada asignación (curso + materia) tiene un programa por año lectivo y, si hace falta, un programa libre (columna tipo); al reemplazarlo, el anterior pasa a PROGRAMAS_VERSIONES. Las cuentas de docentes se vinculan con su nombre en la lista y cada acción queda en REGISTRO.

Diccionario de datos

Los tipos son los de SQLite: INTEGER (número entero) y TEXT (texto). Las fechas y horas se guardan como texto en formato ISO 8601. SQLite no declara las claves foráneas: las relaciones las controla el servidor (FK lógica).

PROGRAMAS — el programa vigente de cada asignación y año. Índice: (asignacion_id, year).

Campo	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	Número del programa (autoincremental)

Campo	Tipo	Nulo	Clave	Descripción
title	TEXT	No	—	Título automático: "Programa de Materia - Curso - Año", o "Programa libre de Materia - Curso - Año" si es el libre
materia	TEXT	No	—	Materia
year	INTEGER	No	—	Año lectivo
curso	TEXT	No	—	Curso y división, por ejemplo 2° 1°
profesor	TEXT	No	—	Docente a cargo
file_url	TEXT	No	—	Dirección para ver el PDF
fecha	TEXT	No	—	Fecha de carga (dd/mm/aaaa)
uploaded_by	TEXT	Sí	—	Cuenta del campus que lo cargó (vacío por defecto)
asignacion_id	TEXT	Sí	FK → ASIGNACIONES.ID	Curso y materia de la lista oficial
almacenamiento	TEXT	Sí	—	Dónde está el PDF: drive o local
archivo_ref	TEXT	Sí	—	Id del archivo en Drive o nombre del archivo en el servidor
created_at	TEXT	Sí	—	Fecha y hora de carga
tipo	TEXT	No	—	regular (programa común, por defecto) o libre. Índice: (asignacion_id, year, tipo)

PROGRAMAS_VERSIONES — las versiones reemplazadas de cada programa. Índice: (programa_id).

Campo	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	Número de la versión (autoincremental)
programa_id	INTEGER	No	FK → PROGRAMAS.id	Programa al que pertenece
title	TEXT	No	—	Título que tenía
profesor	TEXT	No	—	Docente que figuraba
file_url	TEXT	No	—	Dirección del PDF de esa versión
fecha	TEXT	No	—	Fecha de carga de esa versión
uploaded_by	TEXT	No	—	Cuenta que la había cargado
almacenamiento	TEXT	No	—	drive o local

Campo	Tipo	Nulo	Clave	Descripción
archivo_ref	TEXT	No	—	Id en Drive o nombre del archivo
created_at	TEXT	Sí	—	Fecha y hora de carga de esa versión
reemplazado_at	TEXT	No	—	Cuándo se reemplazó
reemplazado_por	TEXT	No	—	Quién la reemplazó

DOCENTES_USUARIOS — la vinculación de cada cuenta de docente con su nombre en la lista oficial.

Campo	Tipo	Nulo	Clave	Descripción
usuario_id	TEXT	No	PK	Id de la cuenta del campus (rol y correo)
nombre_usuario	TEXT	No	—	Nombre de la cuenta en el campus
email	TEXT	No	—	Correo de la cuenta
docente	TEXT	Sí	FK → ASIGNACIONES.Docentes	Nombre elegido en la lista oficial
estado	TEXT	No	—	sin_vincular, pendiente o confirmado
ultimo_acceso	TEXT	Sí	—	Última vez que entró
solicitado_at	TEXT	Sí	—	Cuándo eligió su nombre
confirmado_at	TEXT	Sí	—	Cuándo quedó vinculado
confirmado_por	TEXT	Sí	—	Quién lo vinculó (automático, el docente o la asesora)

REGISTRO — cada acción que modifica datos. Índice: (fecha).

Campo	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	Número de la acción (autoincremental)
fecha	TEXT	No	—	Fecha y hora
usuario_id	TEXT	No	—	Cuenta que hizo la acción, o sistema
usuario_nombre	TEXT	No	—	Nombre de esa cuenta
rol	TEXT	No	—	Rol con el que actuó
accion	TEXT	No	—	Tipo de acción, por ejemplo programa.subido
detalle	TEXT	No	—	Datos de la acción en formato JSON

AJUSTES — configuración que cambia la asesora.

Campo	Tipo	Nulo	Clave	Descripción
clave	TEXT	No	PK	Nombre del ajuste, por ejemplo fecha_limite_2026

Campo	Tipo	Nulo	Clave	Descripción
valor	TEXT	No	—	Valor del ajuste (la fecha límite, AAAA-MM-DD)

ASIGNACIONES — hoja "Asignaciones" de la planilla data/asignaciones.xlsx (no es una tabla de la base).

Campo	Tipo	Nulo	Clave	Descripción
ID	Texto	No	PK	Curso y materia, por ejemplo 2-1_ingles (lo genera el sistema si falta)
Curso	Texto	No	—	Curso y división
Turno	Texto	Sí	—	Turno del curso
Materia	Texto	No	—	Materia
Docentes	Texto	Sí	—	Docentes de la materia, separados por " / "
Requiere programa	Texto	Sí	—	NO si la materia no cuenta como pendiente
Origen	Texto	Sí	—	De qué planilla de horarios salió la fila
Revisar	Texto	Sí	—	Marca del importador para controlar la fila

9. Anexos

Anexo A: Relevamiento y estudio de factibilidad

No se realizaron entrevistas. El relevamiento se hizo a partir del análisis del problema, de las planillas de horarios de la escuela (de donde sale la lista oficial de 473 asignaciones) y de las definiciones que tomó el equipo durante el desarrollo:

- Los alumnos y visitantes consultan los programas sin iniciar sesión.
- Las cuentas llegan verificadas desde el campus virtual de la escuela y no necesitan aprobación. La asesoría entra al campus como directivo.
- Cada materia se controla por curso: varios docentes de una misma materia entregan programas distintos.
- Los alumnos pueden necesitar programas de años anteriores.
- Solo la asesora elimina programas y supervisa los faltantes.
- El diseño debe ser el del sistema de asistencia de la escuela (PROJECT_ASIST).

Estudio de factibilidad

Aspecto	Resultado
Análisis preliminar	Hace falta centralizar y organizar los programas de examen, y controlar cuáles faltan.
Técnica	Viable: HTML, CSS, JavaScript, Node.js, Express y Google Drive son tecnologías conocidas por el equipo y de uso libre.
Económica	Viable: bajo costo; las herramientas son gratuitas y el almacenamiento usa el Google Drive de la escuela.
Operativa	Viable: los alumnos consultan, los docentes cargan y la asesora supervisa, sin cambiar el proceso de la escuela.
Legal	Viable: cumple las normas del marco legal (sección 4).
Social y ambiental	Reduce el uso de papel y facilita el acceso de alumnos y familias a los programas.
Conclusión	El proyecto es viable y recomendable.

Anexo B: Código

El código fuente completo está en el repositorio público github.com/alan4352/progMesas. La tabla resume todos los módulos; a continuación se incluye el código completo de los archivos principales: el puente del campus, la autenticación, la base de datos, los programas y sus versiones, la vinculación de docentes, el almacenamiento y la API.

Archivo	Función
`backend/server.js`	API REST y servidor de archivos (24 rutas)
`backend/config.js`	Configuración desde `.env` y variables de entorno

Archivo	Función
`backend/db.js`	Base SQLite y migraciones
`backend/auth.js`	Identidad del usuario: verifica el pase del campus virtual
`backend/docentes.js`	Vinculación de cuentas con la lista oficial
`backend/asignaciones.js`	Lectura de la lista oficial (`data/asignaciones.xlsx`)
`backend/store.js`	Programas y versiones anteriores
`backend/almacenamiento.js`, `backend/drive.js`	Guardado en Google Drive o en el servidor y paso a Drive
`backend/registro.js`, `backend/ajustes.js`, `backend/respaldos.js`	Registro de actividad, fecha límite y copias de seguridad
`backend/lib/zip.js`	Generación de los ZIP por curso
`index.html`, `frontend/app.js`	Página principal: listado, búsqueda y carga (el ingreso es desde el campus)
`frontend/dashboard.html`, `frontend/dashboard.js`	Dashboard de la asesora
`frontend/styles.css`, `frontend/tema.js`, `frontend/marco.js`	Diseño, modo claro/oscuro y menú lateral
`demo/`	Demo estática para GitHub Pages con la API simulada en el navegador
`tests/`	Pruebas automáticas de la API y de la interfaz

Para el ingreso desde el campus virtual se agregaron al campus (PHP) dos archivos, que también están en el repositorio en `integraciones/campus-escuela/`:

Archivo del campus	Función
`programas.php`	Abre ProgMesas con la sesión del campus; sin sesión, manda al login y vuelve solo
`includes/progmesas.php`	Arma el pase firmado (JWT HS256) con id, nombre, correo y rol
`login.php`, `dashboard.php`, `modulo.php`, `includes/config.php`	Modificados: guardan el correo en la sesión, enlazan el módulo de programas y definen la clave compartida

B.1 Campus: `includes/progmesas.php` (PHP)

Arma el pase firmado con el que el campus abre ProgMesas.

```
<?php
/* ====================================================================
   ARCHIVO: includes/progmesas.php
   PARA QUÉ SIRVE: arma el "pase" (token JWT firmado con HS256) que
   permite entrar a ProgMesas con la sesión del campus, sin volver a
   escribir la contraseña.

   El pase lleva: quién es (id), su nombre, su correo y su rol del campus
   (alumno, profesor o directivo). ProgMesas lo traduce así:
     profesor  → docente   (carga los programas de cualquier curso)
     directivo → asesora   (dashboard y control de entregas)
     alumno    → alumno    (consulta)
   Está firmado con PROGMESAS_SECRETO: si alguien lo modifica, ProgMesas
   lo rechaza.
   ==================================================================== */
require_once __DIR__ . '/config.php';

// Codificación "base64url" que usa JWT (base64 sin '=' y con - y _).
function base64url($datos) {
    return rtrim(strtr(base64_encode($datos), '+/', '-_'), '=');
}

// true si la clave compartida ya se configuró en config.php.
function progmesasConfigurado() {
    return strlen(PROGMESAS_SECRETO) >= 32 && strpos(PROGMESAS_SECRETO,
'CAMBIAR') !== 0;
}

// Crea el pase para el usuario con sesión iniciada.
function crearPaseProgMesas($id, $nombre, $correo, $rol) {
    $cabecera = ['alg' => 'HS256', 'typ' => 'JWT'];
    $ahora = time();
    $datos = [
        'sub'   => (string) $id,
        'name'  => (string) $nombre,
        'email' => (string) $correo,
        'role'  => (string) $rol,
        'iat'   => $ahora,
        'exp'   => $ahora + PROGMESAS_HORAS * 3600,
    ];
    $json = JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES;
    $partes = base64url(json_encode($cabecera, $json)) . '.' .
base64url(json_encode($datos, $json));
    $firma = hash_hmac('sha256', $partes, PROGMESAS_SECRETO, true);
    return $partes . '.' . base64url($firma);
}

// Dirección de ProgMesas con el pase incluido.
function enlaceProgMesas($id, $nombre, $correo, $rol) {
    return rtrim(PROGMESAS_URL, '/') . '/?token=' .
rawurlencode(crearPaseProgMesas($id, $nombre, $correo, $rol));
}
```

B.2 Campus: `programas.php` (PHP)

Módulo "Programas de examen" del campus: abre ProgMesas con la sesión iniciada.

```
<?php
/* ====================================================================
   ARCHIVO: programas.php
   PARA QUÉ SIRVE: abre ProgMesas (programas de examen) con la sesión
```

```
   del campus. Es el destino del módulo "Programas de examen".

   - Sin sesión: manda al login y, al ingresar, vuelve aquí solo.
   - Con sesión: arma el pase y redirige a ProgMesas.
   ========================================================================= */
require_once 'includes/sesion.php';
require_once 'includes/progmesas.php';

if (!estaLogueado()) {
    $_SESSION['volver_a'] = 'programas.php'; // login.php lo usa al
ingresar
    header('Location: login.php');
    exit;
}

// Mismo control que el resto de los módulos (tabla $PERMISOS en
sesion.php)
if (!tienePermiso('programa')) {
    http_response_code(403);
    $error = 'Su tipo de usuario no tiene permiso para ver este módulo.';
} elseif (!progmesasConfigurado()) {
    $error = 'Falta configurar la clave de ProgMesas (PROGMESAS_SECRETO en
includes/config.php).';
} else {
    // El id identifica a la persona en ProgMesas: rol + correo (o DNI /
nombre si no hay correo)
    $correo = $_SESSION['correo'] ?? '';
    $id = $_SESSION['id'] ?? ($_SESSION['rol'] . ':' . ($correo !== '' ?
$correo : $_SESSION['nombre']));
    header('Location: ' . enlaceProgMesas($id, $_SESSION['nombre'],
$correo, $_SESSION['rol']));
    exit;
}
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Programas de examen — Campus Virtual</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<main class="contenedor panel">
    <h1>Programas de examen</h1>
    <div class="error"><?php echo e($error); ?></div>
    <p><a href="dashboard.php" class="boton boton--chico">← Volver al
panel</a></p>
</main>
</body>
</html>
```

B.3 ProgMesas: `backend/auth.js` (JavaScript)

Identidad del usuario: verifica el pase del campus, traduce los roles y controla los permisos. ProgMesas no tiene login propio.

```
// Identidad del usuario. Es la ÚNICA parte de ProgMesas que sabe de dónde
viene el usuario.
// ProgMesas no tiene login propio: la persona inicia sesión en el campus
```

```
de la escuela y llega
// con un pase firmado (token JWT) que se verifica con la clave compartida
(EXTERNO_JWT_SECRET).
//
// El resto del sistema sólo usa req.user = { id, nombre, email, role },
donde role es uno de:
//   teacher (docente) | advisor (asesora; en el campus, directivo) |
viewer (consulta: preceptor) | student (alumno)
// Para conectar otro sistema de ingreso sólo hace falta tocar este archivo
y el .env.

import jwt from 'jsonwebtoken';
import { EXTERNO, ROLES } from './config.js';

// ================ ROLES ================

const PRIORIDAD = ['advisor', 'teacher', 'viewer', 'student'];

/** Traduce el rol del campus (texto o lista) al rol interno. Devuelve null
si no se reconoce. */
export function mapearRol(valor) {
    const valores = (Array.isArray(valor) ? valor : String(valor ??
'').split(','))
        .map((v) => String(v).trim().toLowerCase())
        .filter(Boolean);
    for (const interno of PRIORIDAD) {
        if (valores.some((v) => v === interno ||
ROLES[interno].includes(v))) return interno;
    }
    return null;
}

// ================ LECTURA DEL TOKEN ================

function leerCookie(req, nombre) {
    const cookies = req.headers.cookie || '';
    for (const parte of cookies.split(';')) {
        const [k, ...v] = parte.trim().split('=');
        if (k === nombre) return decodeURIComponent(v.join('='));
    }
    return null;
}

function leerToken(req) {
    const header = req.headers.authorization || '';
    if (header.startsWith('Bearer ')) return header.slice(7);
    if (EXTERNO.cookie) return leerCookie(req, EXTERNO.cookie);
    return null;
}

function valorClaim(payload, ruta) {
    // admite claims anidados, por ejemplo "usuario.id"
    return ruta.split('.').reduce((obj, k) => (obj == null ? undefined :
obj[k]), payload);
}

/**
 * Devuelve la identidad normalizada o lanza un error con status 401.
 * @returns {{ id: string, nombre: string, email: string, role: string }}
 */
```

```
function verificarToken(token) {
    const clave = EXTERNO.clavePublica || EXTERNO.secreto;
    const algoritmos = EXTERNO.clavePublica ? ['RS256', 'RS384', 'RS512',
'ES256', 'ES384'] : ['HS256', 'HS384', 'HS512'];
    const p = jwt.verify(token, clave, { algorithms: algoritmos });

    const id = valorClaim(p, EXTERNO.claimId);
    const role = mapearRol(valorClaim(p, EXTERNO.claimRol));
    if (id === undefined || id === null || id === '') throw new Error(`El
token no trae el dato "${EXTERNO.claimId}"`);
    if (!role) throw new Error('Rol no reconocido');
    return {
        id: String(id),
        nombre: String(valorClaim(p, EXTERNO.claimNombre) || ''),
        email: String(valorClaim(p, EXTERNO.claimEmail) || ''),
        role
    };
}

// ================ MIDDLEWARES ================

/** Exige un usuario identificado. */
export function auth(req, res, next) {
    const token = leerToken(req);
    if (!token) return res.status(401).json({ error: 'Tenés que iniciar
sesión' });
    try {
        req.user = verificarToken(token);
        next();
    } catch (err) {
        const mensaje = err.message === 'Rol no reconocido'
            ? 'Tu rol no tiene acceso a ProgMesas'
            : 'La sesión venció o no es válida. Volvé a ingresar.';
        res.status(401).json({ error: mensaje });
    }
}

/** Si hay un usuario identificado lo usa; si no, sigue como visitante. */
export function authOpcional(req, res, next) {
    const token = leerToken(req);
    req.user = null;
    if (token) {
        try {
            req.user = verificarToken(token);
        } catch {
            req.user = null;
        }
    }
    next();
}

export function rol(...roles) {
    return (req, res, next) => {
        if (!req.user || !roles.includes(req.user.role)) {
            return res.status(403).json({ error: 'No tenés permiso para
esta acción' });
        }
        next();
    };
}
```

B.4 ProgMesas: `backend/db.js` (JavaScript)

Crea la base SQLite, la tabla de programas y las migraciones de versiones anteriores. ProgMesas no guarda usuarios: vienen del campus.

```
import Database from 'better-sqlite3';
import fs from 'fs';
import path from 'path';
import { RUTAS, BACKEND } from './config.js';

const db = new Database(RUTAS.db);
db.pragma('journal_mode = WAL');

// ================ PROGRAMAS ================

db.exec(`
    CREATE TABLE IF NOT EXISTS programas (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        materia TEXT NOT NULL,
        year INTEGER NOT NULL,
        curso TEXT NOT NULL,
        profesor TEXT NOT NULL,
        file_url TEXT NOT NULL,
        fecha TEXT NOT NULL,
        uploaded_by TEXT DEFAULT ''
    )
`);

function asegurarColumna(tabla, columna, definicion) {
    const cols = db.pragma(`table_info(${tabla})`);
    if (!cols.some((c) => c.name === columna)) {
        db.exec(`ALTER TABLE ${tabla} ADD COLUMN ${columna} $
{definicion}`);
        return true;
    }
    return false;
}

asegurarColumna('programas', 'uploaded_by', "TEXT DEFAULT ''");
asegurarColumna('programas', 'asignacion_id', 'TEXT');
asegurarColumna('programas', 'almacenamiento', "TEXT DEFAULT ''"); //
'drive' | 'local'
asegurarColumna('programas', 'archivo_ref', "TEXT DEFAULT ''");    // id de
Drive o nombre del archivo local
const nuevaCreatedAt = asegurarColumna('programas', 'created_at', 'TEXT');

db.exec('CREATE INDEX IF NOT EXISTS idx_programas_asignacion ON programas
(asignacion_id, year)');

// Completar datos de las filas creadas con la versión anterior
if (nuevaCreatedAt) {
    const filas = db.prepare('SELECT id, fecha, file_url FROM
programas').all();
    const actualizar = db.prepare(
        'UPDATE programas SET created_at = ?, file_url = ?, almacenamiento
= ?, archivo_ref = ? WHERE id = ?'
    );
    db.transaction(() => {
```

```
        for (const f of filas) {
            // fecha "27/8/2026" → ISO
            const m = String(f.fecha ||
'').match(/^(\d{1,2})\/(\d{1,2})\/(\d{4})$/);
            const iso = m
                ? new Date(Number(m[3]), Number(m[2]) - 1, Number(m[1]),
12).toISOString()
                : new Date().toISOString();

            let url = f.file_url || '';
            let almacenamiento = '';
            let ref = '';
            const local = url.match(/\/uploads\/([^/?#]+)$/);
            const drive = url.match(/drive\.google\.com\/file\/d\/([^/]
+)/);
            if (local) {
                // "http://localhost:3000/uploads/x.pdf" → "/uploads/x.pdf"
(funciona desde cualquier equipo)
                ref = decodeURIComponent(local[1]);
                url = `/uploads/${encodeURIComponent(ref)}`;
                almacenamiento = 'local';
            } else if (drive) {
                ref = drive[1];
                almacenamiento = 'drive';
            }
            actualizar.run(iso, url, almacenamiento, ref, f.id);
        }
    })();
    console.log(`[DB] Se actualizaron ${filas.length} programas al nuevo
formato`);
}

// Migración única desde el antiguo programas.json
const { cantidad } = db.prepare('SELECT COUNT(*) AS cantidad FROM
programas').get();
const jsonPath = path.join(BACKEND, 'programas.json');
if (cantidad === 0 && fs.existsSync(jsonPath)) {
    try {
        const lista = JSON.parse(fs.readFileSync(jsonPath, 'utf-8'));
        if (Array.isArray(lista) && lista.length > 0) {
            const insertar = db.prepare(`
                INSERT INTO programas (title, materia, year, curso,
profesor, file_url, fecha, uploaded_by, created_at, almacenamiento)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            `);
            db.transaction(() => {
                for (const p of lista) {
                    insertar.run(
                        String(p.title || 'Sin título'),
                        String(p.materia || 'Sin materia'),
                        Number(p.year) || new Date().getFullYear(),
                        String(p.curso || ''),
                        String(p.profesor || 'Sin profesor'),
                        String(p.file_url || ''),
                        String(p.fecha || new
Date().toLocaleDateString('es-AR')),
                        String(p.uploaded_by || ''),
                        new Date().toISOString(),
                        String(p.file_url ||
'').includes('drive.google.com') ? 'drive' : ''
```

```
                    );
                }
            })();
            console.log(`[DB] Migrados ${lista.length} programas desde
programas.json`);
        }
    } catch (err) {
        console.error('[DB] Error al migrar programas.json:', err.message);
    }
}

export default db;
```

B.5 ProgMesas: `backend/store.js` (JavaScript)

Programas cargados y versiones anteriores: alta, reemplazo con historial, reutilización, eliminación y paso de archivos a Drive.

```
import db from './db.js';

// Todas las consultas usan parámetros (?), así los datos del usuario nunca
se interpretan como SQL.

const CAMPOS = `id, title, materia, year, curso, profesor, file_url, fecha,
uploaded_by,
                asignacion_id, almacenamiento, archivo_ref, created_at`;

// ================ VERSIONES ANTERIORES ================
// Al reemplazar un programa, la versión anterior (con su archivo) se
guarda acá en lugar de borrarse.

db.exec(`
    CREATE TABLE IF NOT EXISTS programas_versiones (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        programa_id INTEGER NOT NULL,
        title TEXT NOT NULL,
        profesor TEXT NOT NULL DEFAULT '',
        file_url TEXT NOT NULL,
        fecha TEXT NOT NULL DEFAULT '',
        uploaded_by TEXT NOT NULL DEFAULT '',
        almacenamiento TEXT NOT NULL DEFAULT '',
        archivo_ref TEXT NOT NULL DEFAULT '',
        created_at TEXT,
        reemplazado_at TEXT NOT NULL,
        reemplazado_por TEXT NOT NULL DEFAULT ''
    )
`);
db.exec('CREATE INDEX IF NOT EXISTS idx_versiones_programa ON
programas_versiones (programa_id)');

const stmtTodos = db.prepare(`SELECT ${CAMPOS} FROM programas ORDER BY year
DESC, created_at DESC, id DESC`);
const stmtPorId = db.prepare(`SELECT ${CAMPOS} FROM programas WHERE id = ?
`);
const stmtPorAsignacion = db.prepare(
    `SELECT ${CAMPOS} FROM programas WHERE asignacion_id = ? AND year = ?
ORDER BY created_at DESC, id DESC LIMIT 1`
);
const stmtAnteriorDeAsignacion = db.prepare(
    `SELECT ${CAMPOS} FROM programas WHERE asignacion_id = ? AND year < ?
```

```
ORDER BY year DESC, created_at DESC, id DESC LIMIT 1`
);
const stmtSinAsignacion = db.prepare(`SELECT ${CAMPOS} FROM programas WHERE
asignacion_id IS NULL OR asignacion_id = ''`);

const stmtInsertar = db.prepare(`
    INSERT INTO programas (title, materia, year, curso, profesor, file_url,
fecha, uploaded_by,
                           asignacion_id, almacenamiento, archivo_ref,
created_at)
    VALUES (@title, @materia, @year, @curso, @profesor, @file_url, @fecha,
@uploaded_by,
            @asignacion_id, @almacenamiento, @archivo_ref, @created_at)
`);

const stmtReemplazar = db.prepare(`
    UPDATE programas SET title = @title, materia = @materia, curso =
@curso, profesor = @profesor,
        file_url = @file_url, fecha = @fecha, uploaded_by = @uploaded_by,
        almacenamiento = @almacenamiento, archivo_ref = @archivo_ref,
created_at = @created_at
    WHERE id = @id
`);

const stmtGuardarVersion = db.prepare(`
    INSERT INTO programas_versiones (programa_id, title, profesor,
file_url, fecha, uploaded_by,
                                     almacenamiento, archivo_ref,
created_at, reemplazado_at, reemplazado_por)
    VALUES (@id, @title, @profesor, @file_url, @fecha, @uploaded_by,
            @almacenamiento, @archivo_ref, @created_at, @reemplazado_at,
@reemplazado_por)
`);
const stmtVersiones = db.prepare('SELECT * FROM programas_versiones WHERE
programa_id = ? ORDER BY reemplazado_at DESC, id DESC');
const stmtCantidadVersiones = db.prepare('SELECT programa_id, COUNT(*) AS
cantidad FROM programas_versiones GROUP BY programa_id');
const stmtEliminarVersiones = db.prepare('DELETE FROM programas_versiones
WHERE programa_id = ?');

const stmtVincular = db.prepare('UPDATE programas SET asignacion_id = ?,
curso = ?, materia = ? WHERE id = ?');
const stmtEliminar = db.prepare('DELETE FROM programas WHERE id = ?');

const stmtUsosArchivo = db.prepare(`
    SELECT (SELECT COUNT(*) FROM programas WHERE almacenamiento = @a AND
archivo_ref = @r)
         + (SELECT COUNT(*) FROM programas_versiones WHERE almacenamiento =
@a AND archivo_ref = @r) AS usos
`);

// Archivos guardados en el servidor (para pasarlos a Drive cuando vuelve a
estar disponible)
const stmtArchivosLocales = db.prepare(`
    SELECT archivo_ref, MIN(year) AS year, MIN(curso) AS curso,
MIN(materia) AS materia, MIN(profesor) AS profesor FROM (
        SELECT archivo_ref, year, curso, materia, profesor FROM programas
WHERE almacenamiento = 'local' AND archivo_ref <> ''
        UNION ALL
        SELECT v.archivo_ref, p.year, p.curso, p.materia, v.profesor
```

```
        FROM programas_versiones v JOIN programas p ON p.id = v.programa_id
        WHERE v.almacenamiento = 'local' AND v.archivo_ref <> ''
    ) GROUP BY archivo_ref
`);
const stmtMoverArchivoProgramas = db.prepare(
    "UPDATE programas SET almacenamiento = @nuevo_a, archivo_ref =
@nuevo_r, file_url = @url WHERE almacenamiento = 'local' AND archivo_ref =
@r"
);
const stmtMoverArchivoVersiones = db.prepare(
    "UPDATE programas_versiones SET almacenamiento = @nuevo_a, archivo_ref
= @nuevo_r, file_url = @url WHERE almacenamiento = 'local' AND archivo_ref
= @r"
);

export function getProgramas() {
    return stmtTodos.all();
}

export function getPrograma(id) {
    return stmtPorId.get(Number(id));
}

export function programaDeAsignacion(asignacionId, anio) {
    return stmtPorAsignacion.get(String(asignacionId), Number(anio));
}

/** El programa más reciente de la misma asignación en un año anterior
(para reutilizarlo). */
export function programaAnteriorDeAsignacion(asignacionId, anio) {
    return stmtAnteriorDeAsignacion.get(String(asignacionId),
Number(anio));
}

export function programasSinAsignacion() {
    return stmtSinAsignacion.all();
}

function datosBase(p) {
    const ahora = new Date();
    return {
        title: String(p.title || '').trim(),
        materia: String(p.materia || '').trim(),
        curso: String(p.curso || '').trim(),
        profesor: String(p.profesor || '').trim(),
        file_url: String(p.file_url || '').trim(),
        fecha: ahora.toLocaleDateString('es-AR'),
        uploaded_by: String(p.uploaded_by || '').trim(),
        almacenamiento: p.almacenamiento || '',
        archivo_ref: p.archivo_ref || '',
        created_at: ahora.toISOString()
    };
}

export function agregarPrograma(p) {
    const info = stmtInsertar.run({
        ...datosBase(p),
        year: parseInt(p.year, 10) || new Date().getFullYear(),
        asignacion_id: p.asignacion_id || null
    });
```

```
    return Number(info.lastInsertRowid);
}

/**
 * Reemplaza el archivo y los datos de un programa existente (misma
asignación y año).
 * La versión anterior queda en el historial, con su archivo.
 */
export const reemplazarPrograma = db.transaction((id, p, reemplazadoPor =
'') => {
    const anterior = getPrograma(id);
    if (anterior) {
        stmtGuardarVersion.run({
            ...anterior,
            profesor: anterior.profesor || '',
            uploaded_by: anterior.uploaded_by || '',
            almacenamiento: anterior.almacenamiento || '',
            archivo_ref: anterior.archivo_ref || '',
            fecha: anterior.fecha || '',
            reemplazado_at: new Date().toISOString(),
            reemplazado_por: String(reemplazadoPor || '')
        });
    }
    stmtReemplazar.run({ ...datosBase(p), id: Number(id) });
});

/** Versiones anteriores de un programa, de la más nueva a la más vieja. */
export function versionesDe(programaId) {
    return stmtVersiones.all(Number(programaId));
}

/** Cantidad de versiones anteriores por programa: Map(programa_id →
cantidad). */
export function cantidadDeVersiones() {
    return new Map(stmtCantidadVersiones.all().map((r) => [r.programa_id,
r.cantidad]));
}

/** Asocia un programa a una asignación y unifica el curso y la materia con
los nombres oficiales. */
export function vincularAsignacion(id, asignacion) {
    stmtVincular.run(asignacion.id, asignacion.curso, asignacion.materia,
Number(id));
}

/**
 * Elimina un programa y su historial. Devuelve los registros borrados
(programa y versiones)
 * para que se borren sus archivos, o null si no existía.
 */
export const eliminarPrograma = db.transaction((id) => {
    const programa = getPrograma(id);
    if (!programa) return null;
    const versiones = versionesDe(programa.id);
    stmtEliminarVersiones.run(programa.id);
    stmtEliminar.run(programa.id);
    return [programa, ...versiones];
});

/** Cuántos programas o versiones usan un archivo (un programa reutilizado
```

```
comparte el archivo). */
export function usosDeArchivo(almacenamiento, archivoRef) {
    if (!almacenamiento || !archivoRef) return 0;
    return stmtUsosArchivo.get({ a: almacenamiento, r: archivoRef }).usos;
}

/** Archivos guardados en el servidor, con los datos para ubicarlos en
Drive. */
export function archivosLocales() {
    return stmtArchivosLocales.all();
}

/** Apunta todos los programas y versiones de un archivo local al archivo
nuevo (en Drive). */
export const moverArchivo = db.transaction((archivoRefLocal, nuevo) => {
    const params = { r: archivoRefLocal, nuevo_a: nuevo.almacenamiento,
nuevo_r: nuevo.archivo_ref, url: nuevo.file_url };
    return stmtMoverArchivoProgramas.run(params).changes +
stmtMoverArchivoVersiones.run(params).changes;
});
```

B.6 ProgMesas: `backend/docentes.js` (JavaScript)

Vinculación de la cuenta de cada docente con su nombre en la lista oficial (automática, elegida o corregida por la asesora).

```
// Vinculación entre la cuenta de un docente (ID único que manda el campus)
y su nombre
// en la lista oficial de asignaciones (data/asignaciones.xlsx).
//
// Las cuentas ya vienen verificadas por el campus, así que no hace falta
que la asesora apruebe nada.
// Flujo:
//   1. El docente entra por primera vez → queda registrado como
"sin_vincular".
//      Si su nombre coincide con UNO SOLO de la lista oficial, se vincula
automáticamente.
//   2. Si no, elige su nombre de la lista una vez → "confirmado" y ya
puede subir programas.
// La asesora puede asignar, corregir o desvincular el nombre desde el
dashboard.
// (El estado "pendiente" quedó de la versión anterior, cuando la asesora
tenía que aprobar.)

import db from './db.js';
import { claveTexto } from './lib/normalizar.js';
import { docenteOficial, listaDocentes, asignacionesDeDocente } from
'./asignaciones.js';
import { registrar } from './registro.js';

db.exec(`
    CREATE TABLE IF NOT EXISTS docentes_usuarios (
        usuario_id TEXT PRIMARY KEY,
        nombre_usuario TEXT NOT NULL DEFAULT '',
        email TEXT NOT NULL DEFAULT '',
        docente TEXT,
        estado TEXT NOT NULL DEFAULT 'sin_vincular'
            CHECK (estado IN ('sin_vincular', 'pendiente', 'confirmado')),
        ultimo_acceso TEXT,
        solicitado_at TEXT,
```

```
        confirmado_at TEXT,
        confirmado_por TEXT
    )
`);

const stmtPorId = db.prepare('SELECT * FROM docentes_usuarios WHERE
usuario_id = ?');
const stmtTodos = db.prepare('SELECT * FROM docentes_usuarios ORDER BY
estado, nombre_usuario');
const stmtRegistrar = db.prepare(`
    INSERT INTO docentes_usuarios (usuario_id, nombre_usuario, email,
ultimo_acceso)
    VALUES (@id, @nombre, @email, @ahora)
    ON CONFLICT(usuario_id) DO UPDATE SET
        nombre_usuario = CASE WHEN @nombre <> '' THEN @nombre ELSE
nombre_usuario END,
        email = CASE WHEN @email <> '' THEN @email ELSE email END,
        ultimo_acceso = @ahora
`);
const stmtSolicitar = db.prepare(`
    UPDATE docentes_usuarios SET docente = ?, estado = 'pendiente',
solicitado_at = ?,
        confirmado_at = NULL, confirmado_por = NULL
    WHERE usuario_id = ?
`);
const stmtConfirmar = db.prepare(`
    UPDATE docentes_usuarios SET docente = ?, estado = 'confirmado',
confirmado_at = ?, confirmado_por = ?
    WHERE usuario_id = ?
`);
const stmtDesvincular = db.prepare(`
    UPDATE docentes_usuarios SET docente = NULL, estado = 'sin_vincular',
solicitado_at = NULL,
        confirmado_at = NULL, confirmado_por = NULL
    WHERE usuario_id = ?
`);

// Las solicitudes que quedaron esperando la aprobación de la asesora
(versión anterior) se confirman
db.transaction(() => {
    const pendientes = db.prepare("SELECT * FROM docentes_usuarios WHERE
estado = 'pendiente'").all();
    for (const v of pendientes) {
        const tomado = v.docente && db.prepare(
            "SELECT 1 FROM docentes_usuarios WHERE estado = 'confirmado'
AND usuario_id <> ? AND docente = ?"
        ).get(v.usuario_id, v.docente);
        if (v.docente && !tomado) {
            stmtConfirmar.run(v.docente, v.solicitado_at || new
Date().toISOString(), 'el docente', v.usuario_id);
        } else {
            stmtDesvincular.run(v.usuario_id);
        }
    }
})();

function error(mensaje, status = 400) {
    return Object.assign(new Error(mensaje), { status });
}
```

```
/** Otro usuario que ya tiene confirmado ese nombre (o null). */
function confirmadoPorOtro(docente, usuarioId) {
    const k = claveTexto(docente);
    return stmtTodos.all().find((v) => v.estado === 'confirmado' &&
v.usuario_id !== usuarioId && claveTexto(v.docente) === k) || null;
}

/** Palabras del nombre, sin acentos y en cualquier orden ("Lucrecia
Fernández" = "Fernandez Lucrecia"). */
function clavePalabras(nombre) {
    return claveTexto(nombre).split(' ').filter(Boolean).sort().join(' ');
}

/**
 * Nombre de la lista oficial que corresponde al nombre de la cuenta, sólo
si coincide con uno solo
 * y completo (mismas palabras). Un apellido suelto o un nombre repetido no
se vinculan solos.
 */
export function nombreOficialPara(nombreCuenta) {
    const clave = clavePalabras(nombreCuenta);
    if (!clave.includes(' ')) return null;
    const coincidencias = listaDocentes().filter((d) => clavePalabras(d)
=== clave);
    return coincidencias.length === 1 ? coincidencias[0] : null;
}

/** Registra (o actualiza) a un docente cada vez que entra, y lo vincula
solo si su nombre coincide. */
export function registrarAcceso(usuario) {
    stmtRegistrar.run({ id: usuario.id, nombre: usuario.nombre || '',
email: usuario.email || '', ahora: new Date().toISOString() });
    const v = stmtPorId.get(usuario.id);
    if (v.estado === 'sin_vincular') {
        const docente = nombreOficialPara(usuario.nombre);
        if (docente && !confirmadoPorOtro(docente, usuario.id)) {
            stmtConfirmar.run(docente, new Date().toISOString(),
'automático', usuario.id);
            registrar(usuario, 'docente.vinculado', { docente, por:
'automático' });
            return stmtPorId.get(usuario.id);
        }
    }
    return v;
}

export function vinculoDe(usuarioId) {
    return stmtPorId.get(String(usuarioId)) || null;
}

/** Nombre oficial del docente si el vínculo está confirmado; si no, cadena
vacía. */
export function docenteConfirmado(usuarioId) {
    const v = vinculoDe(usuarioId);
    return v && v.estado === 'confirmado' ? v.docente : '';
}

/** El docente elige su nombre de la lista: queda vinculado en el momento
(la cuenta ya viene verificada). */
export function solicitarVinculo(usuario, nombre) {
```

```
    const v = registrarAcceso(usuario);
    if (v.estado === 'confirmado') {
        throw error('Tu cuenta ya está vinculada. Si el nombre es
incorrecto, pedile a la asesora que lo corrija.', 409);
    }
    const docente = docenteOficial(nombre);
    if (!docente) throw error('Ese nombre no figura en la lista de
asignaciones');
    if (confirmadoPorOtro(docente, usuario.id)) {
        throw error('Ese nombre ya está vinculado a otra cuenta. Hablá con
la asesora.', 409);
    }
    const ahora = new Date().toISOString();
    stmtSolicitar.run(docente, ahora, usuario.id);
    stmtConfirmar.run(docente, ahora, 'el docente', usuario.id);
    return vinculoDe(usuario.id);
}

/** La asesora confirma el nombre pedido, o asigna/corrige uno
directamente. */
export function confirmarVinculo(usuarioId, nombre, asesora) {
    const v = vinculoDe(usuarioId);
    if (!v) throw error('No existe ese usuario', 404);
    const docente = docenteOficial(nombre ?? v.docente);
    if (!docente) throw error('Ese nombre no figura en la lista de
asignaciones');
    const otro = confirmadoPorOtro(docente, usuarioId);
    if (otro) throw error(`"${docente}" ya está vinculado a $
{otro.nombre_usuario || otro.usuario_id}`, 409);
    stmtConfirmar.run(docente, new Date().toISOString(), asesora.nombre ||
asesora.id, usuarioId);
    return vinculoDe(usuarioId);
}

/** Rechazar una solicitud o desvincular una cuenta confirmada. */
export function desvincular(usuarioId) {
    if (stmtDesvincular.run(String(usuarioId)).changes === 0) throw
error('No existe ese usuario', 404);
}

/** Resumen para el dashboard. */
export function resumenVinculos() {
    const usuarios = stmtTodos.all().map((v) => ({
        usuarioId: v.usuario_id,
        nombre: v.nombre_usuario,
        email: v.email,
        docente: v.docente,
        estado: v.estado,
        ultimoAcceso: v.ultimo_acceso,
        solicitado: v.solicitado_at,
        confirmado: v.confirmado_at,
        confirmadoPor: v.confirmado_por,
        asignaciones: v.docente ? asignacionesDeDocente(v.docente).length :
0
    }));
    const confirmados = new Set(usuarios.filter((u) => u.estado ===
'confirmado').map((u) => claveTexto(u.docente)));
    const sinCuenta = listaDocentes()
        .filter((d) => !confirmados.has(claveTexto(d)))
        .map((d) => ({ docente: d, asignaciones:
```

```
asignacionesDeDocente(d).length }));
    return { usuarios, sinCuenta };
}
```

B.7 ProgMesas: `backend/almacenamiento.js` (JavaScript)

Guarda los PDF en Google Drive o, si Drive no responde, en el servidor, y los pasa a Drive cuando vuelve.

```
// Guarda los PDF en Google Drive y, si Drive no está disponible, en la
carpeta uploads del servidor.
// Cuando Drive vuelve, pasarADrive() sube los que quedaron en el servidor.

import fs from 'fs';
import path from 'path';
import { RUTAS } from './config.js';
import { subirArchivo, eliminarArchivo, descargarArchivo, verificarDrive }
from './drive.js';
import { claveTexto } from './lib/normalizar.js';
import { usosDeArchivo, archivosLocales, moverArchivo } from './store.js';

fs.mkdirSync(RUTAS.uploads, { recursive: true });

function nombreSeguro(texto) {
    return claveTexto(texto).replace(/\s+/g, '_').slice(0, 60) ||
'programa';
}

function rutaLocal(archivoRef) {
    // basename evita leer o borrar algo fuera de la carpeta uploads
    return path.join(RUTAS.uploads, path.basename(String(archivoRef ||
'')));
}

/**
 * @returns {Promise<{almacenamiento:'drive'|'local', archivo_ref:string,
file_url:string, aviso:string|null}>}
 */
export async function guardarPdf(buffer, { anio, curso, materia,
profesor }) {
    try {
        const id = await subirArchivo(buffer, anio, materia, curso,
profesor);
        return {
            almacenamiento: 'drive',
            archivo_ref: id,
            file_url: `https://drive.google.com/file/d/${id}/preview`,
            aviso: null
        };
    } catch (err) {
        console.warn('[Almacenamiento] Drive no disponible, se guarda en el
servidor:', err.message);
        const nombre = `${Date.now()}_${nombreSeguro(`${curso} $
{materia}`)}.pdf`;
        fs.writeFileSync(rutaLocal(nombre), buffer);
        return {
            almacenamiento: 'local',
            archivo_ref: nombre,
            file_url: `/uploads/${encodeURIComponent(nombre)}`,
            aviso: `El programa se guardó en el servidor porque Google
```

```
Drive no está disponible (${err.message}).`
        };
    }
}

/** Borra el archivo de un programa o versión, sólo si ningún otro programa
o versión lo usa. */
export async function eliminarPdfSiNoSeUsa(registro) {
    if (!registro?.archivo_ref) return false;
    if (usosDeArchivo(registro.almacenamiento, registro.archivo_ref) > 0)
return false;
    await eliminarPdf(registro);
    return true;
}

export async function eliminarPdf(programa) {
    if (!programa) return;
    if (programa.almacenamiento === 'drive') {
        await eliminarArchivo(programa.archivo_ref);
    } else if (programa.almacenamiento === 'local' && programa.archivo_ref)
{
        fs.rm(rutaLocal(programa.archivo_ref), { force: true }, () => {});
    }
}

/** Contenido del PDF de un programa (local o de Drive). */
export async function leerPdf(programa) {
    if (programa.almacenamiento === 'drive') return
descargarArchivo(programa.archivo_ref);
    if (programa.almacenamiento === 'local') return
fs.promises.readFile(rutaLocal(programa.archivo_ref));
    throw new Error('El programa no tiene un archivo guardado');
}

let sincronizando = null;

/**
 * Sube a Drive los PDF que quedaron guardados en el servidor y actualiza
los programas que los usan.
 * Devuelve { pasados, pendientes, errores }. Si Drive no está disponible
no hace nada.
 */
export function pasarADrive() {
    // Una sola sincronización a la vez
    sincronizando ||= (async () => {
        const locales = archivosLocales();
        const resultado = { pasados: 0, pendientes: locales.length,
errores: [] };
        if (locales.length === 0) return resultado;

        const drive = await verificarDrive();
        if (!drive.disponible) {
            resultado.errores.push(drive.error || 'Google Drive no está
disponible');
            return resultado;
        }

        for (const archivo of locales) {
            try {
                const buffer = await
```

```
fs.promises.readFile(rutaLocal(archivo.archivo_ref));
                const id = await subirArchivo(buffer, archivo.year,
archivo.materia, archivo.curso, archivo.profesor);
                moverArchivo(archivo.archivo_ref, {
                    almacenamiento: 'drive',
                    archivo_ref: id,
                    file_url:
`https://drive.google.com/file/d/${id}/preview`
                });
                fs.rm(rutaLocal(archivo.archivo_ref), { force: true }, ()
=> {});
                resultado.pasados++;
                resultado.pendientes--;
            } catch (err) {
                resultado.errores.push(`${archivo.archivo_ref}: $
{err.message}`);
            }
        }
        if (resultado.pasados) console.log(`[Drive] ${resultado.pasados}
programa(s) guardados en el servidor pasaron a Drive`);
        return resultado;
    })().finally(() => {
        sincronizando = null;
    });
    return sincronizando;
}

/** Cantidad de archivos que están guardados en el servidor (y no en
Drive). */
export function cantidadLocales() {
    return archivosLocales().length;
}
```

B.8 ProgMesas: `backend/server.js` (JavaScript)

API REST y servidor de archivos: seguridad, subida y validación del PDF, vinculación de docentes, programas, control de la asesoría, ZIP por curso y arranque. Por su extensión se presenta en dos partes.

Parte 1 de 2

```
import express from 'express';
import cors from 'cors';
import multer from 'multer';

import { PORT, anioLectivo, rangoDeCarga, CORS_ORIGENES, MAX_PDF_MB, RUTAS,
AUTH_MODO, EXTERNO, TRUST_PROXY } from './config.js';
import { auth, authOpcional, rol } from './auth.js';
import {
    registrarAcceso, vinculoDe, docenteConfirmado, solicitarVinculo,
confirmarVinculo, desvincular, resumenVinculos
} from './docentes.js';
import {
    getProgramas, getPrograma, programaDeAsignacion,
programaAnteriorDeAsignacion, programasSinAsignacion,
    agregarPrograma, reemplazarPrograma, vincularAsignacion,
eliminarPrograma, versionesDe, cantidadDeVersiones
} from './store.js';
import {
    todasLasAsignaciones, asignacionesRequeridas, asignacionPorId,
```

```
asignacionesDeDocente,
    docenteDictaAsignacion, buscarAsignacionPara, estadoAsignaciones,
listaDocentes
} from './asignaciones.js';
import { guardarPdf, eliminarPdfSiNoSeUsa, leerPdf, pasarADrive,
cantidadLocales } from './almacenamiento.js';
import { estadoDrive, verificarDrive } from './drive.js';
import { registrar, ultimosRegistros } from './registro.js';
import { fechaLimite, guardarFechaLimite } from './ajustes.js';
import { programarRespaldos } from './respaldos.js';
import { crearZip } from './lib/zip.js';
import { claveTexto, cursoClave } from './lib/normalizar.js';

const app = express();

// ================= SEGURIDAD BÁSICA =================

app.disable('x-powered-by');
if (TRUST_PROXY !== false) app.set('trust proxy', TRUST_PROXY);
app.use((req, res, next) => {
    res.setHeader('X-Content-Type-Options', 'nosniff');
    res.setHeader('Referrer-Policy', 'same-origin');
    res.setHeader('X-Frame-Options', 'SAMEORIGIN');
    res.setHeader(
        'Content-Security-Policy',
        [
            "default-src 'self'",
            "script-src 'self'",
            "style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com",
            "font-src 'self' https://fonts.gstatic.com",
            "img-src 'self' data:",
            "frame-src 'self' https://drive.google.com",
            "object-src 'none'",
            "base-uri 'self'",
            "form-action 'self'"
        ].join('; ')
    );
    next();
});

// Por defecto el frontend se sirve desde este mismo servidor, así que no
hace falta CORS.
// Si se aloja en otro dominio, configurá CORS_ORIGIN en el .env.
if (CORS_ORIGENES.length) app.use(cors({ origin: CORS_ORIGENES }));

app.use(express.json({ limit: '100kb' }));

// ================= FRONTEND Y ARCHIVOS =================

app.get(['/', '/index.html'], (req, res) => res.sendFile(RUTAS.index));
app.get('/privacidad.html', (req, res) => res.sendFile(RUTAS.privacidad));
app.get('/terminos.html', (req, res) => res.sendFile(RUTAS.terminos));
app.use('/frontend', express.static(RUTAS.frontend));
app.use(
    '/uploads',
    express.static(RUTAS.uploads, {
        setHeaders: (res) => res.setHeader('Content-Type',
'application/pdf')
    })
```

```
);

// ================= SUBIDA (validación del PDF) =================

const upload = multer({
    storage: multer.memoryStorage(),
    limits: { fileSize: MAX_PDF_MB * 1024 * 1024, files: 1, fields: 10 },
    fileFilter: (req, file, cb) => {
        const esPdf = file.mimetype === 'application/pdf' ||
/\.pdf$/i.test(file.originalname || '');
        cb(esPdf ? null : Object.assign(new Error('Sólo se aceptan archivos
PDF'), { status: 400 }), esPdf);
    }
});

function esPdfValido(buffer) {
    return buffer && buffer.length > 5 && buffer.subarray(0,
5).toString('latin1') === '%PDF-';
}

// ================= HELPERS =================

/** Vínculo del docente en formato para el frontend. */
function vinculoPublico(v) {
    return v ? { docente: v.docente, estado: v.estado } : null;
}

/** Quita datos internos antes de mandar un programa a quien no es asesora.
*/
function programaPublico(p, esAsesora) {
    const { uploaded_by, archivo_ref, ...resto } = p;
    return esAsesora ? { ...resto, uploaded_by } : resto;
}

/** El usuario puede ver el historial de un programa: asesora, o docente de
esa materia (o que lo subió). */
function puedeGestionar(usuario, programa) {
    if (usuario?.role === 'advisor') return true;
    if (usuario?.role !== 'teacher') return false;
    const docente = docenteConfirmado(usuario.id);
    const asignacion = programa.asignacion_id ?
asignacionPorId(programa.asignacion_id) : null;
    return (asignacion && docente && docenteDictaAsignacion(docente,
asignacion)) || programa.uploaded_by === usuario.id;
}

/** Título de un programa: siempre se genera solo (curso, materia y año).
*/
function tituloAutomatico(asignacion, anio) {
    return `Programa de ${asignacion.materia} - ${asignacion.curso} - $
{anio}`;
}

/** Nombre de archivo seguro para dentro del ZIP. */
function nombreArchivoZip(texto) {
    return String(texto).replace(/[\\/:*?"<>|\u0000-\u001f]+/g, '
').replace(/\s+/g, ' ').trim().slice(0, 90) || 'programa';
}

/** Vincula los programas cargados antes de esta versión con su asignación
```

```
(por curso + materia). */
function vincularProgramasViejos() {
    let vinculados = 0;
    for (const p of programasSinAsignacion()) {
        const a = buscarAsignacionPara(p.curso, p.materia);
        if (a) {
            vincularAsignacion(p.id, a);
            vinculados++;
        }
    }
    if (vinculados) console.log(`[Programas] ${vinculados} programa(s)
anteriores vinculados a su asignación`);
}

function anioValido(valor) {
    const n = parseInt(valor, 10);
    return Number.isInteger(n) && n >= 2000 && n <= 2100 ? n : null;
}

/** Año para cargar un programa o fijar la fecha límite: sólo el anterior,
el actual o el siguiente. */
function anioDeCarga(valor) {
    const n = anioValido(valor);
    const { minimo, maximo } = rangoDeCarga();
    return n && n >= minimo && n <= maximo ? n : null;
}
function errorAnio() {
    const { minimo, maximo } = rangoDeCarga();
    return { error: `El año lectivo tiene que estar entre ${minimo} y $
{maximo}` };
}

/**
 * Año de un programa que se sube o se reutiliza: siempre el año lectivo
actual.
 * Si el pedido trae otro año, se rechaza (devuelve null).
 */
function anioDelPrograma(valor) {
    const anio = anioLectivo();
    if (valor === undefined || valor === null || valor === '') return anio;
    return anioValido(valor) === anio ? anio : null;
}
function errorAnioPrograma() {
    return { error: `Sólo se pueden cargar programas del año lectivo actual
(${anioLectivo()})` };
}

// ================= API =================

app.get('/api/config', (req, res) => {
    res.json({
        anioLectivo: anioLectivo(),
        anioMinimo: rangoDeCarga().minimo,
        anioMaximo: rangoDeCarga().maximo,
        maxPdfMb: MAX_PDF_MB,
        authModo: AUTH_MODO,
        loginUrl: EXTERNO.loginUrl || null,
        logoutUrl: EXTERNO.logoutUrl || EXTERNO.loginUrl || null,
        fechaLimite: fechaLimite(anioLectivo())
    });
```

```
});

// Quién es el usuario actual. Los docentes quedan registrados para que la
asesora pueda vincularlos.
app.get('/api/yo', authOpcional, (req, res) => {
    if (!req.user) return res.json({ autenticado: false });
    const { id, nombre, email, role } = req.user;
    const vinculo = role === 'teacher' ? registrarAcceso(req.user) : null;
    res.json({ autenticado: true, id, nombre, email, role, vinculo:
vinculoPublico(vinculo) });
});

// ================ VINCULACIÓN DOCENTE ↔ LISTA OFICIAL ================

// Nombres disponibles para que el docente elija el suyo
app.get('/api/docentes/nombres', auth, rol('teacher', 'advisor'), (req,
res) => {
    const tomados = new Set(
        resumenVinculos().usuarios
            .filter((u) => u.estado === 'confirmado' && u.usuarioId !==
req.user.id)
            .map((u) => u.docente)
    );
    res.json(listaDocentes().map((d) => ({ nombre: d, disponible: !
tomados.has(d) })));
});

// El docente elige su nombre (queda vinculado en el momento: la cuenta ya
viene verificada)
app.post('/api/docentes/vinculo', auth, rol('teacher'), (req, res) => {
    const v = solicitarVinculo(req.user, req.body?.docente);
    registrar(req.user, 'docente.vinculado', { docente: v.docente, por: 'el
docente' });
    res.json({ ok: true, vinculo: vinculoPublico(v) });
});

app.get('/api/docentes/vinculos', auth, rol('advisor', 'viewer'), (req,
res) => {
    res.json(resumenVinculos());
});

// La asesora confirma (opcionalmente con otro nombre) o corrige el vínculo
app.post('/api/docentes/vinculos/:usuarioId/confirmar', auth,
rol('advisor'), (req, res) => {
    const antes = vinculoDe(req.params.usuarioId);
    const v = confirmarVinculo(req.params.usuarioId, req.body?.docente,
req.user);
    registrar(req.user, 'docente.corregido', {
        cuenta: v.nombre_usuario || v.usuario_id, docente: v.docente,
anterior: antes?.estado === 'confirmado' ? antes.docente : null
    });
    res.json({ ok: true, vinculo: vinculoPublico(v) });
});

app.delete('/api/docentes/vinculos/:usuarioId', auth, rol('advisor'), (req,
res) => {
    const antes = vinculoDe(req.params.usuarioId);
    desvincular(req.params.usuarioId);
    registrar(req.user, 'docente.desvinculado', { cuenta:
antes?.nombre_usuario || req.params.usuarioId, docente: antes?.docente ||
```

```
null });
    res.json({ ok: true });
});

// Lista de programas: los docentes ven los de sus materias; el resto ve
todos.
app.get('/api/programas', authOpcional, (req, res) => {
    const programas = getProgramas();
    const esAsesora = req.user?.role === 'advisor';
    // Asesora y docentes ven cuántas versiones anteriores tiene cada
programa
    const versiones = esAsesora || req.user?.role === 'teacher' ?
cantidadDeVersiones() : null;
    const conVersiones = (p) => (versiones ? { ...p, versiones:
versiones.get(p.id) || 0 } : p);

    if (req.user?.role === 'teacher') {
        const ids = new
Set(asignacionesDeDocente(docenteConfirmado(req.user.id)).map((a) =>
a.id));
        const propios = programas.filter(
            (p) => ids.has(p.asignacion_id) || p.uploaded_by ===
req.user.id || (req.user.email && p.uploaded_by === req.user.email)
        );
        return res.json(propios.map((p) => conVersiones(programaPublico(p,
false))));
    }

    res.json(programas.map((p) => conVersiones(programaPublico(p,
esAsesora))));
});

// Historial de un programa: la versión actual y las anteriores (asesora o
docente de la materia)
app.get('/api/programas/:id/versiones', auth, rol('teacher', 'advisor'),
(req, res) => {
    const programa = getPrograma(req.params.id);
    if (!programa) return res.status(404).json({ error: 'Programa no
encontrado' });
    if (!puedeGestionar(req.user, programa)) return
res.status(403).json({ error: 'No tenés permiso para esta acción' });
    const esAsesora = req.user.role === 'advisor';
    res.json({
        programa: programaPublico(programa, esAsesora),
        versiones: versionesDe(programa.id).map((v) => ({
            id: v.id,
            title: v.title,
            profesor: v.profesor,
            fecha: v.fecha,
            created_at: v.created_at,
            file_url: v.file_url,
            almacenamiento: v.almacenamiento,
            reemplazado_at: v.reemplazado_at,
            reemplazado_por: v.reemplazado_por,
            ...(esAsesora ? { uploaded_by: v.uploaded_by } : {})
        }))
    });
});

// Asignaciones para el formulario de subida: todas (docentes y asesora
```

```
pueden cargar el programa de
// cualquier curso). A los docentes se les marcan las suyas (propia: true)
para mostrarlas primero.
app.get('/api/asignaciones/mias', auth, rol('teacher', 'advisor'), (req,
res) => {
    const anio = anioValido(req.query.anio) || anioLectivo();
    const esAsesora = req.user.role === 'advisor';
    const docente = esAsesora ? '' : docenteConfirmado(req.user.id);

    const lista = todasLasAsignaciones();
    const propias = new Set(docente ?
asignacionesDeDocente(docente).map((a) => a.id) : []);
    const cargados = new Map();
    const anteriores = new Map(); // el más reciente de un año anterior,
para reutilizarlo
    for (const p of getProgramas()) {
        if (!p.asignacion_id) continue;
        if (p.year === anio && !cargados.has(p.asignacion_id))
cargados.set(p.asignacion_id, p);
        if (p.year < anio && !anteriores.has(p.asignacion_id))
anteriores.set(p.asignacion_id, p);
    }

    res.json({
        anio,
        docente,
        fechaLimite: fechaLimite(anio),
        vinculo: esAsesora ? null : vinculoPublico(vinculoDe(req.user.id)),
        asignaciones: lista.map((a) => {
            const p = cargados.get(a.id);
            const ant = anteriores.get(a.id);
            return {
                id: a.id,
                curso: a.curso,
                turno: a.turno,
                materia: a.materia,
                docentes: a.docentes,
                requiere: a.requiere,
                propia: propias.has(a.id),
                programa: p ? { id: p.id, title: p.title, fecha:
p.fecha } : null,
                anterior: ant ? { id: ant.id, year: ant.year, title:
ant.title, fecha: ant.fecha } : null
            };
        })
    });
});
```

Parte 2 de 2

```
app.post('/api/programas/subir', auth, rol('teacher', 'advisor'),
upload.single('pdf'), async (req, res) => {
    const esAsesora = req.user.role === 'advisor';
    // Docentes y asesora pueden cargar el programa de cualquier curso y
materia.
    // El docente figura con su nombre de la lista (si está vinculado) o
con el de su cuenta del campus.
    const docente = esAsesora ? '' : docenteConfirmado(req.user.id) ||
req.user.nombre || req.user.id;

    const asignacion = asignacionPorId(req.body?.asignacion_id);
```

```
    const anio = anioDelPrograma(req.body?.anio);

    if (!asignacion) return res.status(400).json({ error: 'Elegí la materia
y el curso del programa' });
    if (!anio) return res.status(400).json(errorAnioPrograma());
    if (!req.file) return res.status(400).json({ error: 'Falta el archivo
PDF' });
    if (!esPdfValido(req.file.buffer)) return res.status(400).json({ error:
'El archivo no es un PDF válido' });

    const titulo = tituloAutomatico(asignacion, anio);
    const profesor = esAsesora ? asignacion.docentes.join(' / ') ||
req.user.nombre : docente;

    const archivo = await guardarPdf(req.file.buffer, {
        anio,
        curso: asignacion.curso,
        materia: asignacion.materia,
        profesor
    });

    const datos = {
        title: titulo,
        materia: asignacion.materia,
        year: anio,
        curso: asignacion.curso,
        profesor,
        uploaded_by: req.user.id,
        asignacion_id: asignacion.id,
        ...archivo
    };

    // Si ya había uno para ese curso, materia y año, se reemplaza y el
anterior queda en el historial
    const anterior = programaDeAsignacion(asignacion.id, anio);
    let id;
    if (anterior) {
        reemplazarPrograma(anterior.id, datos, req.user.nombre ||
req.user.id);
        id = anterior.id;
    } else {
        id = agregarPrograma(datos);
    }
    registrar(req.user, anterior ? 'programa.reemplazado' :
'programa.subido', {
        id, curso: asignacion.curso, materia: asignacion.materia, anio,
titulo, almacenamiento: archivo.almacenamiento
    });

    res.json({ ok: true, id, reemplazado: Boolean(anterior),
almacenamiento: archivo.almacenamiento, aviso: archivo.aviso });
});

// Usar para este año el programa de un año anterior (mismo curso y
materia) sin volver a subirlo
app.post('/api/programas/reutilizar', auth, rol('teacher', 'advisor'),
(req, res) => {
    const esAsesora = req.user.role === 'advisor';
    // Docentes y asesora pueden cargar el programa de cualquier curso y
materia.
```

```
    // El docente figura con su nombre de la lista (si está vinculado) o
con el de su cuenta del campus.
    const docente = esAsesora ? '' : docenteConfirmado(req.user.id) ||
req.user.nombre || req.user.id;
    const asignacion = asignacionPorId(req.body?.asignacion_id);
    const anio = anioDelPrograma(req.body?.anio);
    if (!asignacion) return res.status(400).json({ error: 'Elegí la materia
y el curso del programa' });
    if (!anio) return res.status(400).json(errorAnioPrograma());
    const origen = programaAnteriorDeAsignacion(asignacion.id, anio);
    if (!origen) return res.status(404).json({ error: 'No hay un programa
de años anteriores para ese curso y materia' });

    // El archivo se comparte: no se duplica, y sólo se borra cuando ningún
programa lo usa
    const datos = {
        title: tituloAutomatico(asignacion, anio),
        materia: asignacion.materia,
        year: anio,
        curso: asignacion.curso,
        profesor: esAsesora ? origen.profesor || asignacion.docentes.join('
/ ') : docente,
        uploaded_by: req.user.id,
        asignacion_id: asignacion.id,
        file_url: origen.file_url,
        almacenamiento: origen.almacenamiento,
        archivo_ref: origen.archivo_ref
    };
    const actual = programaDeAsignacion(asignacion.id, anio);
    let id;
    if (actual) {
        reemplazarPrograma(actual.id, datos, req.user.nombre ||
req.user.id);
        id = actual.id;
    } else {
        id = agregarPrograma(datos);
    }
    registrar(req.user, 'programa.reutilizado', {
        id, curso: asignacion.curso, materia: asignacion.materia, anio,
desde: origen.year, titulo: datos.title
    });
    res.json({ ok: true, id, reemplazado: Boolean(actual), desde:
origen.year });
});

app.delete('/api/programas/:id', auth, rol('advisor'), async (req, res) =>
{
    const id = parseInt(req.params.id, 10);
    if (isNaN(id)) return res.status(400).json({ error: 'ID inválido' });

    const borrados = eliminarPrograma(id);
    if (!borrados) return res.status(404).json({ error: 'Programa no
encontrado' });
    const [programa] = borrados;

    // Se borran los archivos del programa y de sus versiones, salvo los
que otro programa reutiliza
    for (const r of borrados) await eliminarPdfSiNoSeUsa(r);
    registrar(req.user, 'programa.eliminado', {
        id, curso: programa.curso, materia: programa.materia, anio:
```

```
programa.year, titulo: programa.title,
        versiones: borrados.length - 1
    });
    res.json({ ok: true });
});

// Vincular a mano un programa viejo que no se pudo asociar automáticamente
app.patch('/api/programas/:id/asignacion', auth, rol('advisor'), (req, res)
=> {
    const programa = getPrograma(req.params.id);
    const asignacion = asignacionPorId(req.body?.asignacion_id);
    if (!programa) return res.status(404).json({ error: 'Programa no
encontrado' });
    if (!asignacion) return res.status(400).json({ error: 'Asignación
inexistente' });
    vincularAsignacion(programa.id, asignacion);
    registrar(req.user, 'programa.vinculado', {
        id: programa.id, titulo: programa.title, curso: asignacion.curso,
materia: asignacion.materia, anio: programa.year
    });
    res.json({ ok: true });
});

// ================ CONTROL DE ASESORÍA ================

function calcularCobertura(anio) {
    vincularProgramasViejos();

    const delAnio = getProgramas().filter((p) => p.year === anio);
    const porAsignacion = new Map();
    for (const p of delAnio) {
        if (p.asignacion_id && !porAsignacion.has(p.asignacion_id))
porAsignacion.set(p.asignacion_id, p);
    }

    const asignaciones = asignacionesRequeridas().map((a) => {
        const p = porAsignacion.get(a.id);
        return {
            id: a.id,
            curso: a.curso,
            turno: a.turno,
            materia: a.materia,
            docentes: a.docentes,
            revisar: a.revisar,
            programa: p
                ? { id: p.id, title: p.title, fecha: p.fecha, created_at:
p.created_at, file_url: p.file_url, almacenamiento: p.almacenamiento }
                : null
        };
    });

    const idsValidos = new Set(todasLasAsignaciones().map((a) => a.id));
    const sinAsignacion = delAnio
        .filter((p) => !p.asignacion_id || !
idsValidos.has(p.asignacion_id))
        .map((p) => programaPublico(p, true));

    return { anio, asignaciones, sinAsignacion };
}
```

```
app.get('/api/cobertura', auth, rol('advisor', 'viewer'), (req, res) => {
    const anio = anioValido(req.query.anio) || anioLectivo();
    res.json({
        ...calcularCobertura(anio),
        fechaLimite: fechaLimite(anio),
        estado: { drive: estadoDrive(), asignaciones: estadoAsignaciones(),
locales: cantidadLocales() }
    });
});

// Fecha límite de entrega de un año (la fija la asesora; vacía = sin
fecha)
app.get('/api/fecha-limite', (req, res) => {
    const anio = anioValido(req.query.anio) || anioLectivo();
    res.json({ anio, fecha: fechaLimite(anio) });
});

app.put('/api/fecha-limite', auth, rol('advisor'), (req, res) => {
    const anio = anioDeCarga(req.body?.anio);
    if (!anio) return res.status(400).json(errorAnio());
    const anterior = fechaLimite(anio);
    const fecha = guardarFechaLimite(anio, req.body?.fecha);
    registrar(req.user, 'ajustes.fecha_limite', { anio, fecha, anterior });
    res.json({ ok: true, anio, fecha });
});

// Registro de acciones (asesora y consulta)
app.get('/api/registro', auth, rol('advisor', 'viewer'), (req, res) => {
    res.json(ultimosRegistros(req.query.limite));
});

// Pasar a Drive los programas que quedaron guardados en el servidor
app.post('/api/drive/sincronizar', auth, rol('advisor'), async (req, res)
=> {
    const r = await pasarADrive();
    if (r.pasados) registrar(req.user, 'drive.sincronizado', { pasados:
r.pasados, pendientes: r.pendientes });
    res.json({ ok: r.errores.length === 0, ...r });
});

// Todos los programas de un curso en un ZIP (público, como los programas).
Sin año: todos los años, por carpeta.
app.get('/api/cursos/zip', async (req, res) => {
    const curso = String(req.query.curso || '').trim();
    if (!curso) return res.status(400).json({ error: 'Indicá el curso' });
    const anio = req.query.anio ? anioValido(req.query.anio) : null;
    if (req.query.anio && !anio) return res.status(400).json({ error: 'El
año lectivo no es válido' });

    const clave = cursoClave(curso);
    const programas = getProgramas().filter((p) => cursoClave(p.curso) ===
clave && (!anio || p.year === anio));
    if (programas.length === 0) return res.status(404).json({ error: 'No
hay programas cargados para ese curso' });

    const archivos = [];
    const usados = new Set();
    const faltantes = [];
    for (const p of programas) {
        let nombre = `${p.year}/${nombreArchivoZip(p.materia)}`;
```

```
        for (let i = 2; usados.has(claveTexto(nombre)); i++) nombre = `$
{p.year}/${nombreArchivoZip(p.materia)} (${i})`;
        usados.add(claveTexto(nombre));
        try {
            archivos.push({ nombre: `${nombre}.pdf`, datos: await
leerPdf(p), fecha: new Date(p.created_at || Date.now()) });
        } catch (err) {
            faltantes.push(`${p.year} - ${p.materia}: ${err.message}`);
        }
    }
    if (faltantes.length) {
        archivos.push({
            nombre: 'NO INCLUIDOS.txt',
            datos: Buffer.from('Estos programas no se pudieron descargar:\
r\n\r\n' + faltantes.join('\r\n') + '\r\n', 'utf8')
        });
    }

    const nombreZip = `Programas ${curso}${anio ? ` ${anio}` : ''}.zip`;
    res.setHeader('Content-Type', 'application/zip');
    res.setHeader(
        'Content-Disposition',
        `attachment; filename="${nombreZip.normalize('NFD').replace(/[^\
x20-\x7e]/g, '').replace(/"/g, '')}"; ` +
        `filename*=UTF-8''${encodeURIComponent(nombreZip)}`
    );
    res.send(crearZip(archivos));
});

// Compatibilidad con la versión anterior del dashboard
app.get('/api/programas/faltantes', auth, rol('advisor', 'viewer'), (req,
res) => {
    const anio = anioValido(req.query.anio) || anioLectivo();
    const { asignaciones } = calcularCobertura(anio);
    res.json(
        asignaciones
            .filter((a) => !a.programa)
            .map((a) => ({ id: a.id, curso: a.curso, materia: a.materia,
profesor: a.docentes.join(' / '), turno: a.turno }))
    );
});

app.use('/api', (req, res) => res.status(404).json({ error: 'Ruta
inexistente' }));

// ================= ERRORES =================

// eslint-disable-next-line no-unused-vars
app.use((err, req, res, next) => {
    if (err instanceof multer.MulterError) {
        const mensaje = err.code === 'LIMIT_FILE_SIZE'
            ? `El PDF supera el máximo de ${MAX_PDF_MB} MB`
            : 'No se pudo procesar el archivo enviado';
        return res.status(err.code === 'LIMIT_FILE_SIZE' ? 413 :
400).json({ error: mensaje });
    }
    const status = err.status || err.statusCode || 500;
    if (status >= 500) console.error('[Error]', err);
    res.status(status).json({ error: status >= 500 ? 'Error interno del
servidor' : err.message });
```

```
});

// ================ INICIO ================

vincularProgramasViejos();

// En Express 5 el callback recibe el error si el puerto no se pudo abrir
app.listen(PORT, async (err) => {
    if (err) {
        console.error(
            err.code === 'EADDRINUSE'
                ? `[Servidor] El puerto ${PORT} ya está en uso: hay otro
servidor abierto. Cerralo o cambiá PORT en el .env.`
                : `[Servidor] No se pudo iniciar: ${err.message}`
        );
        process.exit(1);
    }
    console.log(`Servidor corriendo en http://localhost:${PORT}`);
    programarRespaldos((err, r) => registrar(null, err ? 'respaldo.error' :
'respaldo.hecho', err ? { error: err.message } : { carpeta: r.carpeta }));

    const drive = await verificarDrive();
    if (!drive.disponible) console.warn(`[Drive] ${drive.error || 'No
disponible'} — los PDF se guardarán en el servidor`);

    // Cada 6 horas (y ahora) se intenta pasar a Drive lo que quedó
guardado en el servidor
    const sincronizar = () => pasarADrive()
        .then((r) => r.pasados && registrar(null, 'drive.sincronizado',
{ pasados: r.pasados, pendientes: r.pendientes }))
        .catch((err) => console.warn('[Drive] No se pudo sincronizar:',
err.message));
    if (drive.disponible) sincronizar();
    setInterval(sincronizar, 6 * 60 * 60 * 1000).unref();
});
```

Anexo C: Documentación externa (guía de uso)

Esta es la guía de uso completa de ProgMesas para cada rol. Para probar sin instalar nada está la demostración en https://alan4352.github.io/progMesas/ (en la barra amarilla de arriba, Entrar como… hace de campus; lo que se cambia queda sólo en el navegador).

Alumnos, familias y visitantes

No hace falta iniciar sesión. También se puede entrar desde el campus con **Programa de examen**.

1. Entrá a la página principal: aparecen los programas cargados, agrupados por año. Están **todos los años**, así que también se encuentran los programas de materias que se deben de años anteriores.
2. Para encontrar uno, escribí en el buscador (materia, curso, docente o título) o usá los filtros de **materia**, **curso** y **año**.
3. Tocá **Ver PDF** para abrir el programa en el visor de la página. **Cerrar visor** lo oculta.
4. **Compartir:** lo que filtrás queda en la dirección de la página. **Copiar enlace a esta búsqueda** copia esa dirección, por ejemplo para mandar al grupo de un curso el enlace con sus programas.
5. **Descargar todos los de un curso:** elegí el curso (y, si querés, el año) y tocá **Descargar los programas de … (ZIP)**. El ZIP trae una carpeta por año.

Docentes

La primera vez:

1. Iniciá sesión en el **campus** de la escuela con tu código de profesor y abrí **Subir los programas de examen**. ProgMesas no tiene usuario ni contraseña propios: si entrás directo a la página, **Ingresar con el campus** te lleva ahí.
2. Ya podés cargar el programa de **cualquier curso y materia**.
3. Si tu nombre en el campus coincide con el de la lista de horarios, la cuenta se vincula sola: tus materias aparecen primero y figurás con ese nombre. Si no coincide, en **Elegí tu nombre en la lista de docentes (opcional)** escribí tu apellido, elegí tu nombre tal como figura en los horarios y tocá **Confirmar**. Si no lo elegís, los programas que subas figuran con el nombre de tu cuenta del campus. Si te equivocaste, pedile a la asesora que lo corrija.

Cargar un programa:

1. En **Subir programa de examen**, elegí en **Curso y materia** el curso y la materia. Aparecen **todos** los cursos; si estás vinculado, tus materias están primero, en **Mis materias**. Cada curso lleva su propio programa: si das la misma materia en varios cursos, cargá uno por curso. Si es el programa para alumnos libres, marcá «Es el programa libre»: es un programa aparte y no reemplaza al común.
2. El **Año lectivo** es el actual y no se puede cambiar. El título se pone solo (por ejemplo, "Programa de Matemática - 2° 1° - 2026", o "Programa libre de Matemática - 2° 1° - 2026" si es el libre).
3. Elegí el **archivo PDF** (hasta 10 MB) y tocá **Guardar programa**.

Arriba del formulario aparece la **fecha límite** de entrega del año, si la asesora la fijó, y cuántos días faltan.

Usar el programa del año anterior: si para ese curso y materia hay un programa de un año anterior, al elegirlo aparece **Usar el de 2025** (o el año que corresponda). Tocalo y queda cargado para este año sin volver a subir el PDF, con el título de este año.

Reemplazar un programa: subí el nuevo PDF para el mismo curso, materia y año. La página avisa que ya hay uno cargado y, si confirmás, lo reemplaza. La versión anterior **no se borra**: queda en **Versiones anteriores**.

Versiones anteriores: en la lista, los programas que se reemplazaron muestran **Versiones anteriores (n)**. Al tocarlo aparecen las versiones viejas, con quién las reemplazó y cuándo, y se pueden abrir.

En la lista de abajo ves los programas de tus materias y los que subiste vos.

Asesora pedagógica

Entra desde el **campus** como **directivo**, con **Control de programas de examen**. Tiene todo lo de los docentes, para **cualquier** curso y materia, y además puede **Eliminar** programas desde la lista. **Dashboard**, en el menú, abre el panel de control:

- **Año** (en el encabezado, arriba a la derecha): elegí el año lectivo que querés controlar.
- **Avisos** (arriba de todo): si Google Drive no está disponible o hay programas guardados en el servidor y no en Drive. Esos programas pasan solos a Drive cuando vuelve (al arrancar el servidor y cada 6 horas); **Pasar a Drive ahora** lo hace en el momento.
- **Cobertura general:** programas requeridos, entregados, pendientes y porcentaje.

- **Fecha límite:** elegí la fecha y tocá **Guardar fecha** (**Quitar** la borra). Cada año tiene la suya. Los docentes la ven al cargar; cuando pasa, los pendientes se muestran como **Atrasados** y se cuentan los entregados **fuera de término**.
- **Notificaciones y alertas:** asignaciones para revisar en la planilla, los primeros pendientes y los **Programas sin asignación** (programas viejos que no coinciden con ninguna materia oficial): elegí a cuál corresponden y tocá **Vincular**.
- **Estado por curso:** avance de cada curso, con **Copiar enlace para alumnos** (los programas de ese curso, de todos los años) y **Descargar todo (ZIP)**.
- **Estado por docente:** avance de cada docente. A los que tienen pendientes y una cuenta con correo, **Enviar recordatorio** abre un correo ya escrito con lo que les falta y la fecha límite.
- **Programas pendientes:** lista filtrable por docente, materia y curso. **Descargar pendientes (CSV)** la baja para abrir con Excel y **Copiar correos de los docentes con pendientes** copia sus direcciones para escribirles a todos juntos.
- **Cuentas de docentes:** no hace falta aprobar a nadie, porque las cuentas ya vienen verificadas del campus. Esta sección sirve para controlar y corregir con qué nombre de la lista quedó cada uno.
    - *Entró pero no eligió su nombre:* podés escribirlo vos y tocar **Asignar**.
    - *Vinculados:* indica si se vinculó automáticamente, lo eligió el docente o lo corregiste vos. **Cambiar** corrige el nombre y **Desvincular** le quita la posibilidad de cargar programas.
- **Registro de actividad:** quién subió, reemplazó, reutilizó o eliminó programas, quién cambió vinculaciones o la fecha límite, y las tareas automáticas (copias de seguridad, paso a Drive). Se puede buscar por persona, materia, curso o acción.
- **Últimos programas cargados:** los más recientes del año, con su PDF.

Preceptores

Con **Dashboard**, en el menú, ven el mismo panel que la asesora, incluido el registro de actividad, pero sin poder modificar nada (no fijan la fecha límite, no corrigen vinculaciones ni borran programas).

Menú y modo oscuro

Todas las opciones (Programas, Dashboard, Ingresar con el campus, Cerrar sesión, privacidad y condiciones) están en el **menú lateral**, como en el sistema de asistencia de la escuela. En el celular el menú se abre con el botón **Menú** de arriba a la derecha.

La página usa el modo claro u oscuro de la computadora o el celular. **Modo oscuro / Modo claro**, al final del menú, lo cambia y lo recuerda.

Preguntas frecuentes

- **No aparece mi materia en "Mis materias":** tu nombre no figura en esa materia en la lista oficial, o te vinculaste con otro nombre. Igual podés cargarla: buscala en la lista completa, debajo. Si debería ser tuya, pedile a la asesora que lo revise.
- **Me vinculé con el nombre equivocado:** la asesora lo corrige desde **Cuentas de docentes** en el dashboard.
- **"El archivo no es un PDF válido":** el archivo está dañado o no es un PDF aunque termine en .pdf. Exportalo de nuevo como PDF.

- **"Se guardó en el servidor porque Google Drive no está disponible":** el programa quedó guardado igual y pasa solo a Drive cuando vuelve. Si el aviso sigue, hay que renovar el permiso de Drive (ver Anexo D, Google Drive).
- **Reemplacé un programa por error:** el anterior está en **Versiones anteriores**; se puede abrir y volver a subir.
- **¿Quién borró un programa?** Está en el **Registro de actividad** del dashboard.

Anexo D: Documentación interna

Esta es la documentación técnica completa del sistema: funciones, funcionamiento, instalación, configuración, pruebas, despliegue, estructura, seguridad, historial de cambios, instalación en el servidor de la escuela e integración con el campus.

Descripción general

Sistema web para subir, controlar y consultar los **programas de examen** de una escuela secundaria técnica. Los docentes cargan el PDF de cada materia que dictan, los alumnos lo consultan y la asesoría pedagógica ve qué programas faltan.

Proyecto de Práctica Profesional — Escuela Técnica N.° 2 "Independencia".

- **Demo:** https://alan4352.github.io/progMesas/
- **Política de privacidad:** https://alan4352.github.io/progMesas/privacidad.html
- **Condiciones del servicio:** https://alan4352.github.io/progMesas/terminos.html
- **Guía de uso para cada rol:** Anexo C. **Instalación en el servidor de la escuela:** más abajo en este anexo.

Funciones

Rol	Puede
Visitante / alumno	Ver y buscar los programas de todos los años (por materia, curso, año o docente), abrirlos en el visor, compartir el enlace de una búsqueda y descargar todos los de un curso en un ZIP.
Docente	Subir el programa de cualquier curso y materia (las suyas aparecen primero), reemplazarlo (la versión anterior se conserva), reutilizar el del año anterior y ver la fecha límite.
Asesora (en el campus, *directivo*)	Todo lo anterior para cualquier materia, eliminar programas, corregir la vinculación de los docentes, fijar la fecha límite y usar el dashboard.
Consulta (preceptor)	Ver el dashboard y el registro de actividad sin modificar nada.

Programas

- **Programa libre:** cada materia puede tener, además de su programa, un programa libre (para alumnos libres). Al cargarlo se marca **Es el programa libre**; uno no reemplaza al otro. Sólo los libres llevan la etiqueta **Libre** y el título "Programa libre de …"; el filtro permite incluirlos,

sacarlos o ver sólo esos. En el dashboard, la materia cuenta como entregada con cualquiera de los dos.
- **Historial de versiones:** al reemplazar un programa, la versión anterior queda guardada con su PDF, quién la reemplazó y cuándo.
- **Reutilizar:** si un programa no cambió, se usa el del año anterior para el mismo curso y materia sin volver a subirlo (el archivo se comparte, no se duplica).
- **Enlaces por curso:** los filtros quedan en la dirección de la página (`?curso=2° 1°&anio=2026`), así se comparte el enlace de un curso. Sin año, muestra **todos los años**: sirve para alumnos que deben materias de años anteriores.
- **ZIP por curso:** todos los programas de un curso en un archivo, ordenados en carpetas por año; el programa libre va aparte, como "Matemática - Libre.pdf".

Dashboard de asesoría

- Cobertura del año: programas requeridos, entregados, pendientes y porcentaje.
- **Fecha límite** de entrega por año: muestra los atrasados y los entregados fuera de término.
- Avance por curso (con enlace para alumnos y ZIP) y por docente (con **recordatorio por correo** a quien tiene pendientes).
- Lista de pendientes con filtros, exportación a CSV y copia de los correos de los docentes con pendientes.
- **Registro de actividad:** quién subió, reemplazó, reutilizó o eliminó programas, quién cambió vinculaciones o la fecha límite, y las tareas automáticas.
- Avisos cuando Google Drive no está disponible o hay programas guardados en el servidor, con el botón **Pasar a Drive ahora**.
- Control y corrección de la vinculación de las cuentas de docentes.
- Vinculación manual de programas viejos que no coinciden con ninguna materia oficial.

Interfaz

- Mismo diseño que el sistema de asistencia de la escuela (PROJECT_ASIST): fondo bordó con el escudo de marca de agua, menú lateral beige con el escudo, el perfil y las secciones, encabezado con degradé bordó, tarjetas beige con borde bordó y encabezado degradé, tarjetas de estadísticas y tablas con cabecera bordó. En el celular, el menú se abre con el botón **Menú**.
- Modo oscuro como variante del mismo diseño: sigue la preferencia del sistema y se cambia desde el menú (se recuerda).
- Colores en variables al principio de `frontend/styles.css`.
- Íconos SVG propios, sin librerías externas.
- Adaptada a celulares y revisada con axe-core (WCAG 2.1 AA) en los dos modos.

Cómo funciona

- **Integrado al campus de la escuela** — los usuarios inician sesión en el campus virtual (PHP) y entran a ProgMesas desde el módulo de programas de examen: `programas.php` del campus arma un pase firmado (JWT) y ProgMesas lo verifica con la misma clave. ProgMesas **no tiene login propio**. Profesor → docente, directivo → asesora, alumno → alumno. Los archivos y los cambios para el campus se detallan al final de este anexo. Para probar sin el campus, `scripts/token_prueba.js` genera un pase como el del campus.
- **Vinculación de docentes** — las cuentas vienen verificadas del campus virtual. Si el nombre de la cuenta coincide con uno de la lista oficial, se vincula solo; si no, el docente puede

elegirlo. Es opcional: sirve para mostrar sus materias primero y figurar con ese nombre. Sin vincular puede cargar igual (figura con el nombre de su cuenta del campus).
- **Lista oficial de asignaciones** — `backend/data/asignaciones.xlsx` indica qué programa tiene que entregar cada curso (curso + materia + docentes). Se genera a partir de las planillas de horarios y la asesoría la puede editar con Excel: el servidor la relee al guardarla.
- **Cobertura exacta** — cada programa queda asociado a una asignación (curso + materia), así el control no depende de cómo se escribió la materia, y Matemática de 2º 1º y de 3º 2º son programas distintos.
- **Almacenamiento** — los PDF se guardan en Google Drive (carpetas Año / Curso / Materia). Si Drive no está disponible, se guardan en el servidor y, cuando Drive vuelve, **pasan solos a Drive** (al arrancar y cada 6 horas).
- **Copias de seguridad** — con `RESPALDOS_DIR`, el servidor guarda todos los días la base, los PDF del servidor y la planilla, y conserva las últimas copias.

Tecnologías

- **Backend:** Node.js 20+, Express 5, SQLite (better-sqlite3), Multer, JWT, Google Drive API, SheetJS (xlsx 0.20, desde el sitio oficial de SheetJS)
- **Frontend:** HTML, CSS y JavaScript sin frameworks
- **Pruebas:** `node:test`; interfaz con Playwright y axe-core
- **Despliegue:** GitHub Pages (demo), Render (temporal) y servidor de la escuela

Herramientas de desarrollo: Visual Studio Code, Live Server y Node.js 24.15.0.

Instalación

Requisitos: Node.js 20 o superior.

```
npm install        # en la raíz: instala también las dependencias de backend/
```

Creá el archivo `.env` en la raíz del proyecto a partir de `.env.example` y completá `EXTERNO_JWT_SECRET` con la misma clave que `PROGMESAS_SECRETO` del campus (para generar una nueva):

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Iniciá el servidor y abrí http://localhost:3000:

```
npm start
```

Variables de entorno

Variable	Para qué sirve
`EXTERNO_JWT_SECRET`	**Obligatoria.** Clave compartida con el campus (`PROGMESAS_SECRETO`), 32 caracteres o más. Sin ella el servidor no arranca.
`EXTERNO_LOGIN_URL`, `EXTERNO_LOGOUT_URL`	Páginas del campus para ingresar (`programas.php`) y cerrar sesión (`logout.php`).

Variable	Para qué sirve
`ANIO_LECTIVO`	Opcional. El año lectivo se calcula solo (el año en curso en Argentina, cambia el 1 de enero); esta variable lo fija a mano. Los programas se cargan sólo para ese año; la fecha límite admite también el anterior y el siguiente.
`PORT`	Puerto del servidor (3000 por defecto).
`MAX_PDF_MB`	Tamaño máximo de cada PDF (10 por defecto).
`TRUST_PROXY`	`1` si el servidor está detrás de un proxy (Nginx, Render). Hace que los registros usen la IP real del visitante.
`DATOS_DIR`	Carpeta para la base y los PDF locales, por ejemplo un disco persistente. La primera vez se copian ahí los del repositorio.
`RESPALDOS_DIR`	Carpeta donde el servidor guarda una copia de seguridad por día (vacío = sin copias automáticas).
`RESPALDO_HORA`	Hora de la copia diaria (0 a 23; 3 por defecto).
`RESPALDOS_CONSERVAR`	Cuántas copias se conservan (14 por defecto); las más viejas se borran.
`GOOGLE_OAUTH_JSON`, `GOOGLE_TOKEN_JSON`	Contenido de `backend/oauth.json` y `backend/token.json`, para usar Drive en un hosting sin esos archivos.
`DRIVE_CARPETA_RAIZ`	ID de la carpeta de Drive donde se guardan los programas.
`DRIVE_ALCANCE`	`completo` (permiso `drive`, por defecto) o `archivos` (permiso `drive.file`, carpeta propia de la app). Ver Google Drive, más abajo.
`DRIVE_CARPETA_NOMBRE`	Nombre de la carpeta propia con `DRIVE_ALCANCE=archivos`.
`DRIVE_DESACTIVADO`	`1` guarda todo en el servidor aunque haya credenciales.
`DRIVE_SIMULADO`	**Sólo para las pruebas:** carpeta donde un Drive simulado guarda los archivos.
`CORS_ORIGIN`	Otros dominios que pueden usar la API (vacío = sólo el mismo servidor).

Variable	Para qué sirve
Otros `EXTERNO_*`, `ROL_*`	Nombre de los datos dentro del pase y de los roles, sólo para conectar otro sistema de ingreso (ver `.env.example` y Usuarios e ingreso, más abajo).

Google Drive (opcional)

Colocá las credenciales OAuth de Google en `backend/oauth.json` y autorizá la cuenta de la escuela:

```
cd backend
node scripts/autorizar_drive.js
```

Mientras la app de Google esté en modo "Testing", el permiso vence a los 7 días. Para evitarlo, pasala a "En producción" en la pantalla de consentimiento de OAuth (usa las páginas de privacidad y condiciones de arriba).

Pruebas automáticas

```
npm test             # API y funciones: 53 pruebas, unos segundos (las del
campus necesitan PHP)
npm run test:ui      # interfaz en un navegador y accesibilidad: 11 pruebas
```

- **API** (`tests/*.test.js`): utilidades, roles, planilla de asignaciones, ingreso con el pase del campus (y rechazo de pases falsos o vencidos), permisos de cada rol, vinculación de docentes, subida, reemplazo y versiones, reutilizar, fecha límite, registro, ZIP por curso, copias de seguridad, modo externo (pases del campus y de otros sistemas) y Google Drive **simulado** (subida, caída de Drive, paso a Drive cuando vuelve y borrado).
- **Interfaz** (`tests/interfaz.ui.js`): recorre la página en Chromium como visitante, asesora, docente, preceptor y alumno, prueba el modo oscuro y el celular, y revisa la accesibilidad con axe-core en los dos modos. La primera vez hay que instalar el navegador: `npx playwright install chromium`.

Todas usan el servidor real sobre una **copia temporal** de la base y nunca tocan Google Drive ni los datos del proyecto. GitHub Actions las corre en cada cambio (`.github/workflows/pruebas.yml`, Node 20 y 22) y antes de publicar la demo: si alguna falla, no se publica.

Administración

```
cd backend

# Importar las planillas de horarios (una hoja por curso) →
data/asignaciones.xlsx
node scripts/importar_horarios.js "Horarios_T.Mañana.xlsx"
"Horarios_T.Tarde.xlsx"

# Copia de seguridad ahora (base, PDF del servidor y planilla)
node scripts/respaldo.js D:\Respaldos\ProgMesas

# Simular el campus: genera un pase y el enlace para entrar con esa
identidad
node scripts/token_prueba.js --rol profesor --nombre "Valeria Grimberg" --
id 1234
```

ProgMesas no tiene cuentas propias: las personas se dan de alta en el campus. Más detalles en Usuarios e ingreso, más abajo.

Despliegue

Demo en GitHub Pages

https://alan4352.github.io/progMesas/

GitHub Pages sólo sirve archivos estáticos, así que la demo no usa el servidor: `demo/demo.js` atiende las mismas rutas `/api/...` dentro del navegador, con las mismas reglas de permisos, a partir de los datos de la base y de `asignaciones.xlsx`. Los cambios (subir, borrar, vincular docentes, fecha límite) quedan sólo en el navegador de quien la usa y se pueden restablecer. Se publica sola en cada push a `main` (`.github/workflows/pages.yml`), junto con las páginas de privacidad y condiciones. En la demo, el ZIP de un curso no incluye los PDF que están en Google Drive.

En la demo no hay campus: **Entrar como…**, en la barra de demostración, hace de campus y permite entrar como asesora, docente vinculado, docente nuevo (elige su nombre), docente que se vincula sola, preceptor o alumno.

Para probarla en la compu: `npm install`, después `node demo/construir_sitio.js` y servir la carpeta `_site`.

Servidor de la escuela (instalación definitiva)

Los pasos (servicio que arranca solo en Linux o Windows, HTTPS con Nginx, copias de seguridad y actualizaciones) están más abajo, en Instalación en el servidor de la escuela.

Sistema completo en Render (temporal)

Render se usa mientras tanto, hasta instalarlo en el servidor de la escuela. `render.yaml` configura el servidor real: en render.com → **New → Blueprint** → elegir este repositorio. Render vuelve a desplegar en cada push a `main`.

Si el servicio se crea a mano: Build Command `npm install`, Start Command `npm start`, y en **Environment** cargar `EXTERNO_JWT_SECRET, EXTERNO_LOGIN_URL, EXTERNO_LOGOUT_URL` y `TRUST_PROXY=1`. Para Drive, agregar `GOOGLE_OAUTH_JSON` y `GOOGLE_TOKEN_JSON` con el contenido de esos archivos.

En el plan gratis el disco se borra al reiniciar: los PDF guardados en el servidor se pierden. Para conservarlos hace falta Google Drive o un disco persistente (`DATOS_DIR`, ver `render.yaml`).

Estructura

```
├── index.html              Página principal (listado y subida; el ingreso
es desde el campus)
├── privacidad.html         Política de privacidad (pantalla de
consentimiento de Google OAuth)
├── terminos.html           Condiciones del servicio
├── package.json            Arranque desde la raíz (instala backend/ con
postinstall) y scripts de pruebas
├── render.yaml             Despliegue en Render
├── .github/workflows/      Pruebas automáticas y publicación de la demo
en GitHub Pages
├── tests/                  Pruebas automáticas: API (*.test.js), interfaz
(interfaz.ui.js) y servidor de prueba
├── demo/                   Demo estática: datos exportados y API simulada
en el navegador
├── integraciones/
```

```
│   └── campus-escuela/      Puente de ingreso desde el campus de la
escuela (PHP) y cambios a hacer en el campus
├── frontend/
│   ├── styles.css           Estilos y paleta (modo claro y oscuro)
│   ├── tema.js              Cambio de modo claro / oscuro
│   ├── marco.js             Menú lateral (en el celular) y fecha del
encabezado
│   ├── img/escudo.png       Escudo de la escuela (menú y marca de agua)
│   ├── common.js            Utilidades compartidas, sesión, íconos,
descargas y fechas
│   ├── app.js               Página principal
│   └── dashboard.html/.js   Dashboard de la asesora
└── backend/
    ├── server.js            API y servidor de archivos
    ├── config.js            Configuración (.env y variables de entorno)
    ├── db.js                Base SQLite y migraciones
    ├── auth.js              Identidad del usuario (verifica el pase del
campus)
    ├── docentes.js          Vinculación cuenta ↔ nombre en la lista
oficial
    ├── asignaciones.js      Lectura de la lista oficial
    ├── store.js             Programas cargados y versiones anteriores
    ├── registro.js          Registro de actividad
    ├── ajustes.js           Fecha límite por año
    ├── almacenamiento.js    Guardado de PDF (Drive o servidor) y paso a
Drive
    ├── drive.js             Google Drive
    ├── respaldos.js         Copias de seguridad
    ├── lib/                 Importador de horarios, ZIP, Drive simulado y
utilidades
    ├── scripts/             Herramientas de línea de comandos
    ├── data/                asignaciones.xlsx
    └── uploads/             PDF guardados localmente
```

Seguridad

- ProgMesas no guarda contraseñas ni tiene login propio: el pase del campus se verifica con su firma y su vencimiento (se rechazan pases sin firma, firmados con otra clave, vencidos o con roles desconocidos). Las contraseñas las maneja el campus.
- Un docente puede cargar el programa de cualquier curso, pero cada carga queda en el registro de actividad con su cuenta, y si reemplaza un programa la versión anterior se conserva. No puede tomar un nombre de la lista que ya usa otra cuenta. El historial de versiones lo ven sólo la asesora y el docente de la materia.
- Sólo se aceptan PDF válidos (se revisa el contenido, no sólo la extensión; hasta 10 MB por defecto). Los programas se cargan sólo para el año lectivo actual y con título automático (se controla en el servidor, no sólo en el formulario).
- Todo el contenido cargado por usuarios se escapa antes de mostrarse, con una política CSP que bloquea scripts externos.
- Registro de actividad de todas las acciones que modifican datos.
- La exportación a CSV neutraliza textos que Excel interpretaría como fórmulas.
- Las credenciales (`.env`, `oauth.json`, `token.json`, `clave.json`) están excluidas del repositorio; en un hosting se pasan como variables de entorno.
- Dependencias sin vulnerabilidades conocidas (`npm audit`, octubre de 2026).

Historial de cambios

05/11/2026 — Año lectivo fijo y título automático

- El formulario de carga ya no tiene **Título**: siempre se genera solo ("Programa de Materia - Curso - Año"), también al reutilizar el programa de un año anterior. Si un pedido trae otro título, se ignora.
- El **Año lectivo** se muestra fijo (el actual) y no se puede cambiar. El servidor rechaza subir o reutilizar para otro año. La fecha límite la sigue pudiendo fijar la asesora para el año anterior, el actual o el siguiente.
- Pruebas: los programas de años anteriores se cargan directo en la base de prueba (`sembrarPrograma`).

04/11/2026 — El año lectivo se actualiza solo

- El año lectivo es el año en curso en Argentina y cambia solo el 1 de enero, sin tocar el `.env` ni reiniciar el servidor. Con él se corren los años que se pueden cargar (anterior, actual y siguiente) y el año por defecto del formulario y el dashboard.
- `ANIO_LECTIVO` pasa a ser opcional (sólo para fijarlo a mano) y se quitó de `render.yaml` y `.env.example`. La demo también lo calcula.

04/11/2026 — Límite para el año lectivo

- Los programas sólo se pueden cargar (subir o reutilizar) para el año lectivo anterior, el actual o el siguiente (con `ANIO_LECTIVO=2026`: 2025, 2026 o 2027). Lo mismo para fijar la fecha límite. Así nadie carga por error un programa para 2030.
- En el formulario, **Año lectivo** pasa a ser una lista con esos tres años en lugar de un campo libre. El servidor también lo controla.
- Buscar y ver programas de cualquier año sigue igual.

03/11/2026 — Los docentes cargan programas de cualquier curso

- Un docente puede subir o reutilizar el programa de **cualquier curso y materia**, no sólo de los que tiene asignados en la lista de horarios. Sus materias aparecen primero, en **Mis materias**.
- Elegir el nombre de la lista pasa a ser opcional: sin vincular, el programa figura con el nombre de la cuenta del campus.
- Pruebas actualizadas: un docente sube el programa de un curso que no tiene asignado y uno sin vincular también puede subir.

03/11/2026 — Sin login propio: el ingreso es sólo desde el campus

- Se eliminó el login de ProgMesas: el botón **Acceso docentes**, el formulario de correo y contraseña, la ruta `/api/login`, las cuentas de prueba y el script `scripts/usuarios.js`. **Ingresar con el campus** lleva al campus y **Cerrar sesión** cierra también la sesión del campus.
- `EXTERNO_JWT_SECRET` (la clave compartida con el campus) es obligatoria; ya no se usan `AUTH_MODO` ni `JWT_SECRET`.
- Demo: la barra de demostración tiene **Entrar como…** (hace de campus) en lugar de las cuentas con contraseña.
- Pruebas: entran con pases como los del campus; se agregó la comprobación de que la página no pide contraseña.

03/11/2026 — Revisión de la carpeta del proyecto

- Carpeta del proyecto revisada: actores y requerimientos coinciden con el diagrama de casos de uso (alumno, docente, asesora), cantidad de pruebas actualizada y referencias en orden alfabético (APA). El documento incluye completos la guía de uso, la documentación interna y el código de los archivos principales.

02/11/2026 — Ingreso desde el campus de la escuela

- El login viene del campus virtual de la escuela: `integraciones/campus-escuela/programas.php` arma un pase firmado (JWT HS256) con la sesión del campus y abre ProgMesas sin pedir otra contraseña. Si no hay sesión, manda al login del campus y vuelve solo.
- Roles: profesor → docente, **directivo → asesora** (antes, consulta), alumno → alumno.
- Nueva variable `EXTERNO_LOGOUT_URL`: **Cerrar sesión** (en la página principal y en el dashboard) cierra también la sesión del campus.
- La carpeta del proyecto (`Carpeta del Proyecto — ProgMesas.docx`) describe el ingreso desde el campus virtual.
- Pruebas `tests/campus.test.js`: generan pases con el código PHP del campus y comprueban roles, configuración y el rechazo de pases con otra clave o modificados.

27/10/2026 — Diseño del sistema principal

- Las páginas usan el mismo diseño que el sistema de asistencia de la escuela (PROJECT_ASIST): menú lateral beige con escudo, perfil y secciones; encabezado con degradé bordó; fondo bordó con el escudo de marca de agua; tarjetas beige con encabezado degradé; tarjetas de estadísticas con ícono; tablas con cabecera bordó.
- En el celular: encabezado con escudo y botón **Menú** que abre el menú lateral (se cierra con Escape o tocando afuera).
- El botón de modo oscuro y los enlaces a privacidad y condiciones pasan al menú lateral.
- Enlace "Saltar al contenido" para teclado y lectores de pantalla.

27/10/2026 — Mejoras de uso, control y mantenimiento

- Historial de versiones de cada programa (la versión reemplazada ya no se borra).
- Reutilizar el programa del año anterior para el mismo curso y materia.
- Fecha límite de entrega por año, con atrasados y entregados fuera de término en el dashboard.
- Recordatorio por correo a los docentes con pendientes y copia de sus correos.
- Enlaces por curso (filtros en la dirección, con todos los años) y descarga de los programas de un curso en un ZIP.
- Registro de actividad en el dashboard.
- Los PDF guardados en el servidor pasan solos a Google Drive cuando vuelve a estar disponible (o con un botón).
- Copias de seguridad diarias (`RESPALDOS_DIR`) y `scripts/respaldo.js`.
- Modo externo: `scripts/token_prueba.js` para simular el sistema principal, y pruebas automáticas del ingreso con token.
- Pruebas: 51 de API (con Google Drive simulado) y 11 de interfaz y accesibilidad con Playwright y axe-core, todas en GitHub Actions antes de publicar.
- Accesibilidad: mejor contraste de los colores de estado y barras de avance con nombre para lectores de pantalla.

- Arreglo: las copias de seguridad hechas en el mismo segundo no se borraban al pasar el límite.

27/10/2026 — Vinculación de docentes sin aprobación

- La asesora ya no tiene que aprobar a los docentes (las cuentas vienen verificadas del sistema principal):
    - vinculación automática cuando el nombre de la cuenta coincide con uno solo de la lista de horarios;
    - si no coincide, el docente elige su nombre y puede cargar en el momento;
    - la asesora sigue pudiendo asignar, cambiar o desvincular nombres; las solicitudes pendientes se confirmaron solas.
- Demo: cuenta `docente3@test.com` para ver la vinculación automática.
- Política de privacidad y condiciones del servicio actualizadas.

27/10/2026 — Pruebas automáticas y guía de uso

- Pruebas automáticas (`npm test`) de utilidades y de la API con cada rol, corridas por GitHub Actions en cada cambio y antes de publicar la demo.
- Opción `DRIVE_ALCANCE=archivos` para probar el permiso reducido `drive.file` de Google Drive (carpeta propia de la app).
- `DRIVE_DESACTIVADO` para guardar todo en el servidor aunque haya credenciales.
- LEEME: guía de uso para cada rol (Anexo C), preguntas frecuentes e instalación en el servidor de la escuela (systemd / NSSM, Nginx con HTTPS, copias de seguridad y actualizaciones).

27/10/2026 — Publicación, diseño y seguridad

- Demo pública en GitHub Pages con la API simulada en el navegador y publicación automática.
- Despliegue del sistema completo en Render (`render.yaml`, `DATOS_DIR`, credenciales de Drive por variables de entorno).
- Páginas de política de privacidad y condiciones del servicio, enlazadas desde el pie.
- Íconos SVG en lugar de emojis.
- Nueva paleta institucional y modo oscuro.
- Mejoras de seguridad: dependencias actualizadas (Express, Multer y SheetJS sin vulnerabilidades conocidas), `TRUST_PROXY` para el límite de intentos detrás de un proxy, limpieza del registro de intentos y CSV sin fórmulas.
- Arreglos: los filtros ya no se aplastan en el celular y el botón de ver contraseña queda centrado.

API REST

Método y ruta	Uso	Permiso
`GET /api/config`, `GET /api/yo`	Configuración (con las direcciones del campus) y usuario actual	Público / con el pase del campus
`GET /api/programas`	Listado y búsqueda de programas	Público

Método y ruta	Uso	Permiso
`GET /api/programas/:id/versiones`	Versiones anteriores	Asesora o docente de la materia
`GET /api/asignaciones/mias`	Asignaciones del docente	Docente
`POST /api/programas/subir`	Subir o reemplazar un programa	Docente, Asesora
`POST /api/programas/reutilizar`	Reutilizar el del año anterior	Docente, Asesora
`DELETE /api/programas/:id`	Eliminar un programa	Asesora
`PATCH /api/programas/:id/asignacion`	Vincular un programa viejo	Asesora
`GET /api/docentes/nombres`, `POST /api/docentes/vinculo`	Elegir el nombre en la lista	Docente
`GET /api/docentes/vinculos`, `POST .../confirmar`, `DELETE .../:usuarioId`	Controlar vinculaciones	Asesora
`GET /api/cobertura`, `GET /api/programas/faltantes`	Datos del dashboard	Asesora, Consulta
`GET` / `PUT /api/fecha-limite`	Ver o fijar la fecha límite	Público / Asesora
`GET /api/registro`	Registro de actividad	Asesora, Consulta
`POST /api/drive/sincronizar`	Pasar a Drive ahora	Asesora
`GET /api/cursos/zip`	ZIP de un curso	Público

No hay ruta de inicio de sesión: el pase llega del campus en la dirección (`/?token=...`) y se manda en el encabezado `Authorization` de cada pedido.

Instalación en el servidor de la escuela

Render se usa sólo de forma temporal. Para la instalación definitiva en un servidor propio:

Requisitos

- **Node.js 20 o superior** (https://nodejs.org) y **Git**.
- Que el servidor sea accesible desde la red de la escuela (o desde internet, si se quiere publicar afuera).

Instalar

```
git clone https://github.com/alan4352/progMesas.git
cd progMesas
npm install
```

Copiá `.env.example` como `.env` y completalo. Lo mínimo:

```
EXTERNO_JWT_SECRET=<la misma clave que PROGMESAS_SECRETO del campus>
EXTERNO_LOGIN_URL=https://<campus>/programas.php
EXTERNO_LOGOUT_URL=https://<campus>/logout.php
PORT=3000
```

Sin `EXTERNO_JWT_SECRET` el servidor no arranca. Para generar una clave nueva: `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"` (ver Usuarios e ingreso).

Copiá `backend/oauth.json` y generá `backend/token.json` para Google Drive (ver Google Drive).

Comprobá que todo funcione y arrancá:

```
npm test
npm start
```

Abrí `http://<ip-del-servidor>:3000`.

Dejarlo funcionando siempre

El servidor tiene que arrancar solo cuando se prende la máquina.

Linux (systemd). Creá `/etc/systemd/system/progmesas.service` (ajustá la carpeta y el usuario):

```
[Unit]
Description=ProgMesas
After=network.target

[Service]
WorkingDirectory=/opt/progMesas
ExecStart=/usr/bin/node backend/server.js
Restart=always
User=progmesas
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now progmesas
sudo systemctl status progmesas      # ver si está andando
journalctl -u progmesas -f           # ver los mensajes
```

Windows. Con NSSM (https://nssm.cc): `nssm install ProgMesas`, en *Path* poné la ruta de `node.exe`, en *Startup directory* la carpeta del proyecto y en *Arguments* `backend\server.js`. Después `nssm start ProgMesas`.

Publicarlo con HTTPS (recomendado)

Si la página va a estar en un dominio (por ejemplo `programas.escuela.edu.ar`), conviene ponerla detrás de Nginx con un certificado de Let's Encrypt. Ejemplo de Nginx:

```
server {
    server_name programas.escuela.edu.ar;
    client_max_body_size 12m;            # un poco más que MAX_PDF_MB
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Con un proxy adelante, agregá `TRUST_PROXY=1` al `.env` (así los registros usan la IP real de cada visitante). El certificado se saca con `sudo certbot --nginx -d programas.escuela.edu.ar`.

Copias de seguridad

Automáticas: agregá al `.env` la carpeta donde guardarlas (mejor en otro disco):

```
RESPALDOS_DIR=/var/respaldos/progmesas      # en Windows, por ejemplo D:\
Respaldos\ProgMesas
RESPALDO_HORA=3                             # hora de la copia diaria (3 de
la mañana)
RESPALDOS_CONSERVAR=14                      # cuántas copias guardar; las
más viejas se borran
```

Con eso el servidor hace una copia por día, en una carpeta con la fecha y la hora, con:

- `database.sqlite` — programas, versiones, cuentas de docentes, fecha límite y registro de actividad (se copia de forma segura con el servidor andando);
- `uploads/` — PDF guardados en el servidor (los que están en Drive ya quedan en Drive);
- `asignaciones.xlsx` — la lista oficial.

Cada copia queda anotada en el **Registro de actividad** del dashboard.

A mano, en cualquier momento: `cd backend` y `node scripts/respaldo.js` (o `node scripts/respaldo.js <carpeta>`).

Además, guardá una vez en un lugar seguro `.env`, `backend/oauth.json` y `backend/token.json`, que no entran en la copia porque tienen claves.

Restaurar: detené el servidor, copiá `database.sqlite`, `uploads/` y `asignaciones.xlsx` de la copia a `backend/` (y `backend/data/` para la planilla; borrá antes `database.sqlite-wal` y `database.sqlite-shm` si existen), y volvé a arrancarlo. Si se usa `DATOS_DIR`, la base y `uploads` van en esa carpeta.

Actualizar a una versión nueva

```
git pull
npm install
npm test
sudo systemctl restart progmesas      # o reiniciar el servicio de Windows
```

Lista oficial de asignaciones

`backend/data/asignaciones.xlsx` es la lista de qué programa tiene que entregar cada curso (curso + materia + docentes). El sistema controla las entregas con esa lista. Cada **curso + materia** es una asignación distinta: Matemática de 2° 1° y Matemática de 3° 2° son dos programas diferentes, aunque los dé el mismo docente. Para regenerarla con horarios nuevos:

```
cd backend
node scripts/importar_horarios.js "ruta\Horarios_T.Mañana.xlsx" "ruta\
Horarios_T.Tarde.xlsx"
```

Si `asignaciones.xlsx` ya existe, el importador no lo pisa: genera `asignaciones_importadas_<fecha>.xlsx` (con `--reemplazar` lo pisa).

Se puede editar con Excel: el servidor lo relee al guardarlo. Las filas en amarillo (columna *Revisar*) conviene controlarlas: son docentes del contraturno que el importador ubicó por cercanía o materias sin docente. Para que una materia no cuente como pendiente, poné `NO` en *Requiere programa*.

Usuarios e ingreso

ProgMesas forma parte de un sistema más grande: **los usuarios y el inicio de sesión vienen del campus virtual de la escuela**. **ProgMesas no tiene login propio**: no guarda usuarios ni contraseñas. El usuario llega con un pase firmado (JWT) del campus, en el link (`/?token=...`). Todo lo relacionado está en un solo archivo, `backend/auth.js`, y se configura en el `.env` con `EXTERNO_JWT_SECRET` (obligatoria), `EXTERNO_LOGIN_URL` y `EXTERNO_LOGOUT_URL`.

Roles: **docente** (carga sus programas), **asesora** (todo), **consulta** (preceptor: ve el dashboard sin modificar nada) y **alumno** (ve los programas). En el campus de la escuela, la asesoría entra como **directivo**.

Ingreso desde el campus de la escuela

El sistema principal de la escuela es el **campus virtual** (PHP). Los usuarios inician sesión ahí y entran a ProgMesas desde el módulo **Programas de examen**, sin volver a escribir la contraseña:

En el campus	En ProgMesas
Profesor (código de profesor)	Docente: carga los programas de cualquier curso (los suyos aparecen primero)
Directivo (correo y contraseña)	Asesora: dashboard, fecha límite, vinculaciones, registro
Alumno (DNI y contraseña)	Alumno: consulta (también puede entrar sin sesión)

Cómo funciona: `programas.php` del campus arma un pase firmado (JWT HS256) con el nombre, el correo y el rol, y abre ProgMesas con `/?token=...`. ProgMesas lo verifica con la misma clave. El pase dura 8 horas (`PROGMESAS_HORAS`). **Ingresar** en ProgMesas lleva al campus, y **Cerrar sesión** cierra también la sesión del campus.

Para instalarlo:

1. Copiá al campus `integraciones/campus-escuela/programas.php` y `integraciones/campus-escuela/includes/progmesas.php`, y hacé los cambios de configuración, login, panel y módulo que se detallan en Integración con el campus, al final de este anexo.
2. Generá una clave: `php -r "echo bin2hex(random_bytes(32));"`. Ponela en `includes/config.php` del campus (`PROGMESAS_SECRETO`) y la dirección de ProgMesas en `PROGMESAS_URL`.
3. En el `.env` de ProgMesas, con la misma clave (ver el bloque de abajo).
4. Reiniciá ProgMesas. Las pruebas `tests/campus.test.js` generan pases con el mismo código PHP y comprueban los roles y que un pase con otra clave o modificado se rechace.

```
EXTERNO_JWT_SECRET=<la misma clave>
EXTERNO_LOGIN_URL=https://<campus>/programas.php
EXTERNO_LOGOUT_URL=https://<campus>/logout.php
```

Vinculación de docentes

El campus ya verifica las cuentas, pero no sabe qué materias dicta cada docente. Por eso cada cuenta se une con un nombre de `asignaciones.xlsx`:

1. **Automática:** al entrar, si el nombre de la cuenta tiene las mismas palabras que **un solo** nombre de la lista (sin importar el orden ni los acentos: "Valeria Grimberg" = "Grimberg

Valeria"), se vincula sola. Un apellido suelto, un nombre repetido o un nombre que ya usa otra cuenta no se vinculan solos.
2. **Elegida por el docente:** si no hubo coincidencia, el docente elige su nombre una vez y queda vinculado en el momento. No puede elegir un nombre que ya tiene otra cuenta.
3. **Corrección:** la asesora puede asignar, cambiar o desvincular nombres desde **Cuentas de docentes**.

Las solicitudes que habían quedado esperando aprobación con la versión anterior se confirman solas al arrancar el servidor.

Probar sin el campus

En una computadora sin el campus instalado, `scripts/token_prueba.js` hace de campus: genera un pase firmado como el que mandaría y el enlace para entrar con esa identidad.

1. En el `.env`: `EXTERNO_JWT_SECRET=<una clave larga>` (en la escuela, la misma que `PROGMESAS_SECRETO` del campus).
2. Arrancá el servidor y generá un token (ver el bloque de abajo).
3. Abrí el enlace que muestra. Con ese nombre, el docente se vincula solo con "Grimberg Valeria". Probá también `--rol asesora`, `--rol preceptor` o `--rol alumno`.

```
cd backend
node scripts/token_prueba.js --rol profesor --nombre "Valeria Grimberg" --
id 1234 --email vgrimberg@escuela.edu.ar
```

Conectar otro sistema de ingreso

Con el campus ya está todo resuelto (ver arriba). Si en el futuro se conecta otro sistema de ingreso, hacen falta estos datos (se cargan en el `.env`, sin tocar código):

Dato	Variable
Cómo firma el token: clave compartida (HS256) o clave pública (RS256)	`EXTERNO_JWT_SECRET` o `EXTERNO_JWT_CLAVE_PUBLICA`
Cómo llega el token: en el enlace (`/?token=`), en el encabezado `Authorization` o en una cookie (¿cómo se llama?)	`EXTERNO_COOKIE`
Nombre del campo con el identificador único del usuario (por ejemplo `sub` o `usuario.id`)	`EXTERNO_CLAIM_ID`
Nombre del campo con el **nombre completo** (para la vinculación automática)	`EXTERNO_CLAIM_NOMBRE`
Nombre del campo con el rol, y cómo se llaman los roles (docente, asesora, preceptor, directivo, alumno)	`EXTERNO_CLAIM_ROL`, `ROL_*`
Nombre del campo con el correo (para los recordatorios)	`EXTERNO_CLAIM_EMAIL`
Dirección de la página de ingreso	`EXTERNO_LOGIN_URL`
Dirección que cierra la sesión	`EXTERNO_LOGOUT_URL`

Las pruebas automáticas (`tests/externo.test.js`) ya comprueban este modo: roles, campos anidados, token en cookie, vinculación automática y rechazo de tokens con otra firma, vencidos, sin firma o con roles desconocidos.

Google Drive

Los PDF se guardan en Google Drive, en carpetas Año / Curso / Materia. Si Drive no está disponible, se guardan en `backend/uploads` y el dashboard lo avisa. Cuando Drive vuelve, esos PDF **pasan solos a Drive** (al arrancar el servidor y cada 6 horas) y se borran del servidor; la asesora también puede hacerlo con **Pasar a Drive ahora**.

Autorizar o renovar el permiso

```
cd backend
node scripts/autorizar_drive.js
```

Abrí el enlace que aparece, iniciá sesión con la cuenta de la escuela, aceptá y pegá la dirección a la que te redirige. Se guarda `backend/token.json`; después reiniciá el servidor.

Para que no venza cada 7 días, en Google Cloud Console → *Pantalla de consentimiento de OAuth* pasá la app de "Testing" a "En producción". La política de privacidad y las condiciones del servicio ya están publicadas (`/privacidad.html` y `/terminos.html`).

Probar el permiso reducido (`drive.file`)

Hoy la app pide acceso a **todo** el Drive (`drive`), un permiso que Google considera "restringido": para pasar la app a producción sin advertencias pide una evaluación de seguridad. Con `drive.file` la app sólo ve los archivos que crea ella misma, y Google no lo considera restringido. Para probarlo:

1. En Google Cloud Console → *Pantalla de consentimiento de OAuth* → *Acceso a los datos*, agregá el permiso `.../auth/drive.file` (y quitá `.../auth/drive`).
2. En el `.env` agregá `DRIVE_ALCANCE=archivos`.
3. Volvé a autorizar: `cd backend` y `node scripts/autorizar_drive.js` (muestra qué permiso pide).
4. Reiniciá el servidor y subí un programa de prueba. En "Mi unidad" tiene que aparecer la carpeta **ProgMesas - Programas de examen** con las subcarpetas Año / Curso / Materia (el nombre se cambia con `DRIVE_CARPETA_NOMBRE`).

Con este permiso la app no ve la carpeta anterior: los programas viejos siguen abriéndose, pero si se reemplazan o eliminan desde la página, su archivo en Drive hay que borrarlo a mano. Para volver atrás: `DRIVE_ALCANCE=completo` y volver a autorizar.

Archivos que ya no se usan

Se pueden borrar: `backend/excel.js`, `backend/procesar_profesores.js`, `backend/exportar_profesores_excel.js`, `backend/profesores_extraidos.json`, `backend/profesores_extraidos.xlsx`, `backend/horarios_extraidos.json`, `backend/horarios_profes_2026.json`, `horarios_profes_2026.json`, `backend/materias_profesores_editable.xlsx`, `backend/clave.json` (cuenta de servicio sin uso) y `backend/programas.json` (ya migrado a la base).

Integración con el campus: archivos y cambios

ProgMesas no tiene login propio en la escuela: usa el del **campus virtual** (PHP). Estos son los archivos y cambios que se agregan al campus para que, desde el módulo **Programas de examen**, cada persona entre a ProgMesas ya identificada.

En el campus	En ProgMesas
Profesor	Docente: carga los programas de cualquier curso; los suyos aparecen primero (se vincula solo si su nombre coincide con la lista de horarios)
Directivo	Asesora: dashboard, fecha límite, vinculaciones, registro
Alumno	Alumno: consulta

Cómo funciona

1. La persona inicia sesión en el campus como siempre.
2. Al tocar el módulo de programas, `programas.php` arma un **pase** (token JWT firmado con HS256) con su id, nombre, correo y rol, y la manda a `PROGMESAS_URL/?token=...`.
3. ProgMesas verifica la firma con la misma clave (`EXTERNO_JWT_SECRET`). Si alguien cambia el pase o lo firma con otra clave, lo rechaza. El pase vence a las `PROGMESAS_HORAS` horas (8 por defecto).
4. **Ingresar** en ProgMesas lleva a `programas.php` (si no hay sesión, al login del campus, que vuelve solo). **Cerrar sesión** lleva a `logout.php` del campus.

Archivos nuevos (copiar al campus)

Archivo	Para qué
`includes/progmesas.php`	Arma el pase firmado y el enlace a ProgMesas
`programas.php`	Abre ProgMesas con la sesión del campus (o manda al login y vuelve)

El código completo de los dos está en el Anexo B.

Cambios en archivos del campus

`includes/config.php` — agregar al final:

```
define('PROGMESAS_URL', 'https://programas.escuela.edu.ar');   // dirección
de ProgMesas
define('PROGMESAS_SECRETO', 'CAMBIAR-por-una-clave-larga-y-secreta'); //
php -r "echo bin2hex(random_bytes(32));"
define('PROGMESAS_HORAS', 8);
```

Mientras la clave empiece con `CAMBIAR` o tenga menos de 32 caracteres, `programas.php` muestra un aviso y no abre nada.

`login.php` — guardar el correo y un id en la sesión, y volver al módulo que se estaba abriendo:

```
// después de los require_once
function destinoTrasLogin() {
    $destino = $_SESSION['volver_a'] ?? 'dashboard.php';
    unset($_SESSION['volver_a']);
    return preg_match('/^[a-z_]+\.php$/', $destino) ? $destino :
'dashboard.php';
```

```
}

// alumno, al ingresar
$_SESSION['correo'] = $alumno['correo'];
$_SESSION['id']     = 'alumno:' . $alumno['dni'];
header('Location: ' . destinoTrasLogin());

// profesor
$_SESSION['correo'] = $profesor['correo'];
$_SESSION['id']     = 'profesor:' . strtolower($profesor['correo']);
header('Location: ' . destinoTrasLogin());

// directivo
$_SESSION['correo'] = $directivo['correo'];
$_SESSION['id']     = 'directivo:' . strtolower($directivo['correo']);
header('Location: ' . destinoTrasLogin());
```

`dashboard.php` — los módulos de programas apuntan a `programas.php`, y el directivo tiene el suyo:

```
if ($rol === 'directivo') { ?>
    <a class="modulo" href="programas.php">
        <h2>Control de programas de examen</h2>
        <p>Vea qué programas faltan por curso y por docente, fije la fecha
límite y revise el registro.</p>
    </a>
<?php } elseif ($rol === 'profesor') { ?>
    <a class="modulo" href="programas.php"> … Subir los programas de examen
… </a>
<?php } elseif ($rol === 'alumno') { ?>
    <a class="modulo" href="programas.php"> … Programa de examen … </a>
<?php } ?>
```

`modulo.php` — quien entre por `modulo.php?m=programa` va a ProgMesas:

```
$modulo = $_GET['m'] ?? '';
if ($modulo === 'programa') {
    header('Location: programas.php');
    exit;
}
```

Configuración de ProgMesas (`.env`)

```
EXTERNO_JWT_SECRET=<la misma clave que PROGMESAS_SECRETO>
EXTERNO_LOGIN_URL=https://<campus>/programas.php
EXTERNO_LOGOUT_URL=https://<campus>/logout.php
```

Los nombres de los datos (`sub`, `name`, `email`, `role`) y de los roles (`profesor`, `directivo`, `alumno`) ya son los que ProgMesas espera por defecto: no hace falta configurar nada más.

Probar

`npm test` en ProgMesas incluye `tests/campus.test.js`, que genera pases con este mismo `progmesas.php` (hace falta PHP instalado) y comprueba los roles y que un pase con otra clave o modificado se rechace.

10. Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Buzzatto, A., y Rodriguez, V. (2026). *ProgMesas* [Código fuente]. GitHub. https://github.com/alan4352/progMesas

Google. (s.f.). *Google Drive API*. Google for Developers. https://developers.google.com/drive/api

Ley N.° 11.723. (1933). *Régimen legal de la propiedad intelectual*. Boletín Oficial de la República Argentina.

Ley N.° 25.326. (2000). *Protección de los datos personales*. Boletín Oficial de la República Argentina.

Ley N.° 26.206. (2006). *Ley de Educación Nacional*. Boletín Oficial de la República Argentina.

Ley N.° 26.653. (2010). *Acceso a la información pública: accesibilidad de la información en las páginas web*. Boletín Oficial de la República Argentina.

moshi-azz. (s.f.). *PROJECT_ASIST* [Código fuente]. GitHub. https://github.com/moshi-azz/PROJECT_ASIST

OpenJS Foundation. (s.f.-a). *Express*. https://expressjs.com/

OpenJS Foundation. (s.f.-b). *Node.js*. https://nodejs.org/

SQLite. (s.f.). *SQLite*. https://www.sqlite.org/

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/